

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Informatyki

Praca dyplomowa magisterska

na kierunku Informatyka
w specjalności Informatyka w multimediach

Zastosowanie głębokich sieci neuronowych
w detekcji chorób roślin

Jakub Piotr Kostrzewa

Numer albumu

promotor
Prof. Przemysław Rokita

WARSZAWA 2026

Zastosowanie głębokich sieci neuronowych w detekcji chorób roślin

Streszczenie. Celem pracy magisterskiej było zbadanie wybranych modeli głębokiego uczenia w zadaniu klasyfikacji wieloklasowej obrazów roślin oraz określenie wpływu ilości i jakości danych treningowych na uzyskiwane wyniki. W ramach badania przeanalizowano dziesięć modeli należących do pięciu rodzin architektur: ResNet, Vision Transformer(ViT), DenseNet, MobileNet, EfficientNet.

Przeprowadzono dwa eksperymenty w ramach, których modele były szkolone na różnych zbiorach danych, z użyciem techniki transfer learning. Wszystkie modele były wstępnie przetrenowane na tym samym zbiorze. W ramach pierwszego eksperymentu zbadano wpływ ograniczenia ilości danych na jakość klasyfikacji, a w drugim wpływ jakości zbioru treningowego. Jako pierwotny zbiór danych został wykorzystany zbiór PlantVillage-Dataset. Do każdego z eksperymentów przygotowano oddzielne zbiory. Do analizy wyników wykorzystano miarę Macro-F1 Score.

Uzyskane Wyniki wskazały, że większość analizowanych modeli osiąga wysokie wartości miary Macro-F1 w tym zadaniu. Jedynym wyjątkiem były modele DenseNet, które osiągnęły najgorsze wyniki w całym zestawieniu, zdecydowanie niższe od reszty. Modele oparte o transformery nie uzyskały przewagi nad modelami CNN. Najlepszymi modelami okazały się modele z rodzin MobileNet i EfficientNet. Wyniki wskazują na potencjał zbadanych modeli do praktycznego wykorzystania.

Słowa kluczowe: uczenie głębokie, sieci neuronowe, klasyfikacja danych, choroby roślin

Application of Deep Neural Networks in Plant Disease Detection

Abstract. The aim of this master's thesis was to investigate selected deep learning models for multiclass classification of plant images and determine the impact of the quantity and quality of training data on the obtained results. The study analyzed ten models belonging to five architecture families: ResNet, Vision Transformer (ViT), DenseNet, MobileNet, and EfficientNet.

Two experiments were conducted in which the models were trained on different datasets using transfer learning. All models were pre-trained on the same dataset. The first experiment examined the impact of data limitation on classification quality, while the second examined the impact of training set quality. The PlantVillage-Dataset was used as the initial dataset. Separate datasets were prepared for each experiment. The Macro-F1 Score was used to analyze the results.

The obtained results indicated that most of the analyzed models achieved high Macro-F1 scores in this task. The only exception were the DenseNet models, which achieved the worst results in the entire comparison, significantly lower than the rest. Transformer-based models did not demonstrate an advantage over CNN models. The best models were those from the MobileNet and EfficientNet families. The results demonstrate the potential of the models tested for practical use.

Keywords: deep learning, neural networks, data classification, plant diseases



.....
miejscowość i data

.....
imię i nazwisko studenta

.....
numer albumu

.....
kierunek studiów

OŚWIADCZENIE

Świadomy/-a odpowiedzialności karnej za składanie fałszywych zeznań oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie, pod opieką kierującego pracą dyplomową.

Jednocześnie oświadczam, że:

- niniejsza praca dyplomowa nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz.U. z 2006 r. Nr 90, poz. 631 z późn. zm.) oraz dóbr osobistych chronionych prawem cywilnym,
- niniejsza praca dyplomowa nie zawiera danych i informacji, które uzyskałem/-am w sposób niedozwolony,
- niniejsza praca dyplomowa nie była wcześniej podstawą żadnej innej urzędowej procedury związanej z nadawaniem dyplomów lub tytułów zawodowych,
- wszystkie informacje umieszczone w niniejszej pracy, uzyskane ze źródeł pisanych i elektronicznych, zostały udokumentowane w wykazie literatury odpowiednimi odnośnikami,
- znam regulacje prawne Politechniki Warszawskiej w sprawie zarządzania prawami autorskimi i prawami pokrewnymi, prawami własności przemysłowej oraz zasadami komercjalizacji.

Oświadczam, że treść pracy dyplomowej w wersji drukowanej, treść pracy dyplomowej zawartej na nośniku elektronicznym (płycie kompaktowej) oraz treść pracy dyplomowej w module APD systemu USOS są identyczne.

.....
czytelny podpis studenta

Spis treści

1. Wstęp	11
1.1. Sformowanie problemu	11
1.2. Cel i zakres pracy	11
1.3. Układ pracy	11
2. Przegląd literatury	13
2.1. Przegląd badań	13
2.2. Sztuczna inteligencja w rolnictwie	13
2.3. Bazy danych	14
2.3.1. PlantDoc: A Dataset for Visual Plant Disease Detection	14
2.3.2. PlantVillage Dataset	15
2.3.3. New Plant Diseases Dataset	15
2.4. Przykłady aplikacji	16
2.4.1. Planta: Plant & Garden Care	16
2.4.2. Agrio - Plant diagnosis app	17
2.4.3. PlantNet Plant Identification	18
3. Wprowadzenie do nauczania maszynowego	20
3.1. Podstawowe pojęcia	20
3.1.1. Uczenie maszynowe	20
3.1.2. Rodzaje uczenia	20
3.1.3. Klasyfikacja	20
3.1.4. Balans klas	21
3.1.5. Rodzaje zbiorów	21
3.1.6. Generalizacja	21
3.1.7. Przeuczenie	21
3.1.8. Regularyzacja	21
3.1.9. Uczenie transferowe	21
3.1.10. Dostrajanie modelu	21
3.2. Klasyczne metody uczenia maszynowego	22
3.2.1. Maszyna wektorów nośnych - SVM	22
3.2.2. Regresja liniowa	23
3.2.3. Drzewa decyzyjne	23
3.2.4. Algorytm najbliższych sąsiadów - k-NN	24
3.3. Sztuczne sieci neuronowe - ANN	25
3.3.1. Perceptron	26
3.3.2. Perceptron wielowarstwowy - MLP	27
3.4. Uczenie głębokie - Deep learning	28
3.4.1. Konwolucyjne sieci neuronowe - CNN	28

3.4.2. Transformer	29
3.4.3. Rekurencyjne sieci neuronowe - RNN	30
3.5. Metryki ewaluacji wyników	31
3.5.1. Macierz pomyłek	31
3.5.2. Dokładność	32
3.5.3. Precyzja	32
3.5.4. Czułość	33
3.5.5. Miara F1	33
4. Modele	34
4.1. MobileNet	34
4.2. Vision transformer - ViT	35
4.3. ResNet	37
4.4. EfficientNet	38
4.5. DenseNet	40
5. Zbiór danych	41
5.1. Charakterystyka zbioru danych	41
5.2. Analiza zestawu	42
6. Metodyka badań	43
6.1. Środowisko badawcze	43
6.2. Lista modeli wykorzystana w badaniu	44
6.3. Podział zbiorów danych	44
6.3.1. Dane z różnym procentem uszkodzonych danych	45
6.4. Procedura treningu	46
6.5. Konfiguracja treningu	46
6.5.1. Augmentacja danych	47
6.5.2. Parametry	48
6.5.3. Wczesne zatrzymywanie	49
6.5.4. Precyzja mieszana	50
6.6. Wykonane eksperymenty	50
6.6.1. Trenowanie modeli na zbiorach z różną liczbą danych	50
6.6.2. Trenowanie modeli na zbiorach z różnym procentem uszkodzonych danych	50
6.7. Użyte miary	51
7. Wyniki	52
7.1. Charakterystyka badanych modeli	52
7.2. Eksperyment 1 - Skalowanie efektywności względem ilości danych treningowych	52
7.3. Eksperyment 2 - Skalowanie efektywności względem ilości zaszumionych danych	53

8. Analiza wyników	54
8.1. Analiza wpływu danych	54
8.2. Analiza wpływu zanieczyszczeń w danych	55
8.3. Finalna ocena modeli	56
9. Aplikacja	58
9.1. Technologie	58
9.2. Implementacja modelu	58
9.3. Widoki i działanie	59
10. Podsumowanie	60
10.1. Dalsze kierunki badań	61
Bibliografia	63
Wykaz symboli i skrótów	66
Spis rysunków	66
Spis tabel	67
Spis załączników	67

1. Wstęp

1.1. Sformowanie problemu

W ostatnich latach można zaobserwować dynamiczny rozwój technologii sztucznej inteligencji. Technologia ta znajduje coraz szersze zastosowanie w różnych gałęziach przemysłu, od branży IT aż po medycynę. Narzędzia te wykorzystywane są do automatyzacji procesów oraz wspierania człowieka w wykonywanej pracy. Przemysł rolniczy nie jest w tej kwestii wyjątkiem - także w tej gałęzi gospodarki wprowadzane są nowoczesne technologie, nie będące tylko nowym modelem traktora, lecz szerokim zakresem produktów opartych na uczeniu maszynowym.

Choroby oraz szkodniki to jedne z głównych przyczyn śmierci roślin uprawnych - niszczą one prawie 40% globalnych zbiorów żywności[1]. Przekłada się to na wielomiliardowe straty. Poza aspektem ekonomicznym, warto zwrócić uwagę na aspekt społeczny - w samej Polsce, stan na 2025 rok, nawet 1,4 miliona mieszkańców zmaga się z głodem [2]. Straty spowodowane chorobami zwiększają ceny żywności w sklepach, pogłębiając problem dostępu do jedzenia, szczególnie dla osób ubogich. W związku z tym ważne jest opracowanie i zbadanie rozwiązań detekcji chorób roślin, w celu zmniejszenia strat oraz zapobiegania masowemu rozprzestrzenianiu się chorób.

1.2. Cel i zakres pracy

Celem tej pracy jest zbadanie, jak wybrane architektury głębokiego uczenia różnią się pod względem skuteczności oraz odporności na redukcję liczby danych treningowych i degradację ich jakości w zadaniu klasyfikacji chorób roślin. Dodatkowym celem jest stworzenie aplikacji mobilnej, stanowiącej przykład praktycznej implementacji badanej technologii. W ramach pracy opracowano badania umożliwiające ocenę dostępnych publicznie modeli. Przygotowano także demo technologiczne.

1.3. Układ pracy

Praca zawiera dziesięć rozdziałów. Rozdział pierwszy to wstęp zawierający sformułowanie problemu, cel i zakres pracy.

Rozdział drugi przedstawia przegląd literatury, prezentując badania w temacie, zastosowanie sztucznej inteligencji w rolnictwie, zbiory danych zawierające zdjęcia oraz przykłady aplikacji, które służą do detekcji chorób roślin.

Rozdział trzeci jest wprowadzeniem do uczenia maszynowego. Przedstawia podstawowe pojęcia potrzebne do zrozumienia pracy oraz przegląd technik uczenia maszynowego.

Rozdział czwarty poświęcony jest opisowi wybranych architektur poddanych analizie. Rozdział piąty przedstawia wybrany zbiór danych do badań.

W ramach rozdziału szóstego przedstawiono opracowane badanie. Opisano użyte techniki oraz scenariusze eksperymentów.

Rozdział siódmy przedstawia wyniki eksperymentów.

Rozdział ósmy poświęcony jest analizie uzyskanych wyników.

Rozdział dziewiąty przedstawia wykonane demo technologiczne oraz wykorzystane do tego technologie.

Ostatni, dziesiąty rozdział stanowi podsumowanie całej pracy oraz przedstawia możliwe kierunki rozwoju badań.

2. Przegląd literatury

W niniejszym rozdziale przedstawiono przegląd literatury w zakresie wykrywania chorób roślin przy użyciu klasyfikacji obrazów. Przedstawiono inne badania, przeprowadzonych z wykorzystaniem różnych algorytmów uczenia głębokiego. Opisano również dostępne publicznie zbiory danych wykorzystywane w badaniach z użyciem uczenia maszynowego, spośród których wybrano zbiór wykorzystany w niniejszej pracy. Zbiory te zawierają zdjęcia liści roślin zdrowych, jak liści dotkniętych różnymi chorobami, głównie gatunki hodowane przez rolników, takie jak pomidory, czy ziemniak. Przedstawiono także przegląd zastosowań metod sztucznej inteligencji w rolnictwie oraz przykłady aplikacji.

2.1. Przegląd badań

Wraz z rozwojem technologii uczenia maszynowego, problem rozpoznania roślin, czy identyfikacji chorób ich trapiących z wykorzystaniem tych metod wzrasta na popularności. Skutkiem tego jest rosnąca liczba artykułów naukowych, prac i badań poruszających ten temat. Literatura naukowa w tematyce detekcji chorób roślin wykorzystuje głównie zbiór danych PlantVillage, który zawiera rośliny rolnicze. W badaniu autorstwa Sharada P.Mohanty, David P.Hughes oraz Marcel Salathé porównano dwa typy architektur uczenia głębokiego: GoogLeNet i AlexNet, w różnych konfiguracjach zmiennych oraz danych. Analiza opierała się głównie na różnych wariantach tego samego zbioru danych. Modele trenowane były z użyciem trzech zestawów tych samych zdjęć - jeden zestaw zawierał kolorowe zdjęcia, drugi zdjęcia w odcieniach szarości, a w trzecim dokonano segmentacji obiektu liścia. Wyniki badania sugerują lepszą skuteczność modelu przy użyciu pełnego kolorowego obrazu, choć różnica między trzema zbiorami danych jest minimalna [3]. W artykule opublikowanym w AgriEngineering autorzy wykorzystują tylko zdjęcia pomidora. Następnie, po przeprowadzonej segmentacji, autorzy zbadali skalowanie modelu EfficientNet w zależności od liczby klas, które ma rozpoznać - od prostego podziału binarnego na zdrowe, bądź chore, do rozpoznania konkretnych chorób [4]. Przedstawione badania pozwalają porównać skuteczność różnych architektur. Należy jednak zauważyć, że używania tego samego zbioru danych, sposób ich użycia, jak i przygotowania do badania różni się między autorami, co nie pozwala na uzyskanie spójnych wniosków dotyczących wyboru najlepszego typu architektury co do rozwiązania tego problemu. Warto także zwrócić uwagę na fakt, że liczba architektur wykorzystana w tych badaniach jest dosyć ograniczona.

2.2. Sztuczna inteligencja w rolnictwie

Metody sztucznej inteligencji (ang. Artificial Intelligence - AI) znajdują szerokie zastosowanie w rolnictwie. Przemysł rolniczy, jako jedna z najważniejszych gałęzi światowej gospodarki, aktywnie rozwija się w celu optymalizacji produkcji żywności, wykorzystując

w tym celu, między innymi sztuczną inteligencję. Sztuczna inteligencja znajduje zastosowanie w wielu obszarach działalności rolniczej. SI jest wykorzystywane do prognozowania i szacowania wielkości plonów. Na podstawie zdjęć modele są w stanie określić wielkość plonów zdalnych do zebrania. Niektóre takie rozwiązania są w stanie określić, czy owoce na danym krzaku są zdalne do spożycia i zainicjować proces zbierania. Niektóre rozwiązania zbierają dane z pól i tworzą raport dla rolnika na temat dojrzałości zbiorów, co pozwala mu na lepsze zaplanowanie sezonu. Inne rozwiązania wykorzystują dane pogodowe do oceny stopnia dojrzałości upraw. SI wykorzystywane jest także w ramach klasyfikacji obrazów. Istnieje wiele rozwiązań, które dokonują detekcji chorób roślin na podstawie zdjęć, między innymi zdjęć liści. Klasyfikacja stosowana jest też w detekcji chwastów i szkodników. SI wykorzystywana jest także w ocenie jakości plonów. Znajduje także zastosowanie w zarządzaniu chowem, gdzie na podstawie np. zachowań krów dokonuje klasyfikacji ich zachowania. Istnieją także rozwiązania, które pozwalają zaoszczędzić wodę - modele na podstawie danych ze szklarni, np. wilgotność ziemi dokonuje decyzji, czy należy podlać rośliny. Sztuczna inteligencja więc nie tylko pozwala na zoptymalizowanie procesu produkcji żywności pochodzenia roślinnego i zwierzęcego, lecz pozwala także na oszczędzanie zasobów takich jak woda. Ma to szczególne znaczenie w czasach globalnego ocieplenia oraz susz, szczególnie w Polsce, gdzie woda staje się zasobem, który należy oszczędzać. [5]

2.3. Bazy danych

Poniżej przedstawiono zbiory danych zawierające zdjęcia liści zdrowych jak i chorych roślin. Zbiory danych przedstawiają głównie rośliny uprawne, hodowane na całym świecie takie jak pomidor, czy kukurydza.

2.3.1. PlantDoc: A Dataset for Visual Plant Disease Detection

CroppedPlant-Doc to zbiór danych zawierający zdjęcia roślin zdrowych oraz chorych. Posiada ponad 2,500 zdjęć 13 gatunków roślin i 17 klas chorób. Wykorzystany został w pracy przedstawionej na konferencji ACM India Joint International Conference on Data Science and Management of Data. Zdjęcia zostały wykonane w naturalnym środowisku, a na wielu z nich widoczny jest także fragment oryginalnej rośliny [6].



Rysunek 2.1. Przykładowe zdjęcia ze zbioru danych PlantDoc. [6]

2.3.2. PlantVillage Dataset

Zbiór danych zawierający 15 klas na temat 3 roślin: ziemniak, pomidor oraz papryka. Każda roślina ma klasę przedstawiającą jej liście w zdrowym stadium oraz kilka klas przedstawiających różne choroby oraz ich objawy na strukturze liści. Zbiór zawiera łącznie ponad 20 tysięcy zdjęć. Został stworzony do konkursu na stronie AICrowd[7].



Rysunek 2.2. Przykładowe zdjęcia ze zbioru danych PlantVillage.[7]

2.3.3. New Plant Diseases Dataset

Zbiór danych z serwisu Kaggle zawierający ponad 87 tysięcy zdjęć roślin podzielonych w 38 różnych klas określających gatunek oraz chorobę rośliny. Dane podzielone są na zestawy treningowe oraz walidacyjne, w stosunku 80/20. Jest to zaktualizowana wersja zbioru danych PlantVillage - prócz oryginalnych zdjęć zawiera dodatkowe gatunki oraz zdjęcia [8].



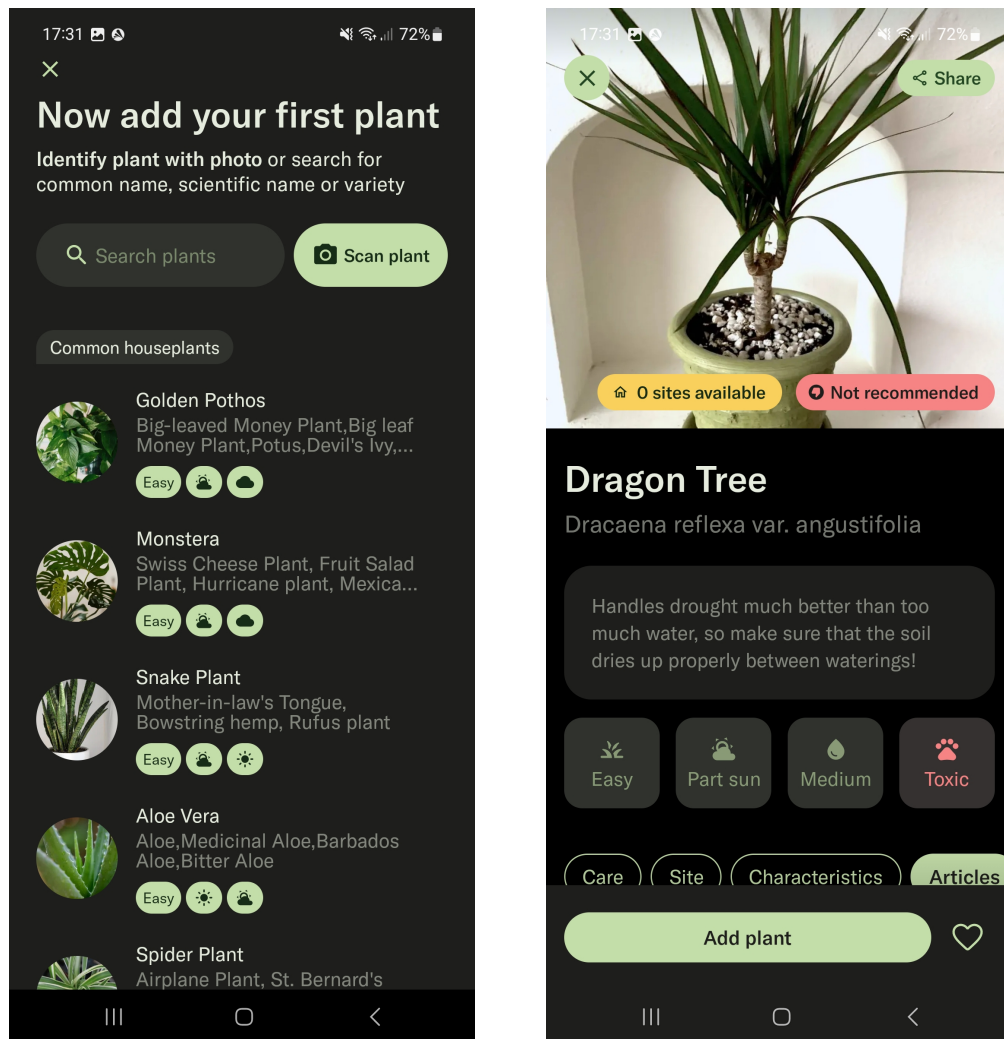
Rysunek 2.3. Przykładowe zdjęcia ze zbioru danych New Plant Diseases.[8]

2.4. Przykłady aplikacji

Poniżej przedstawione aplikacje to rozwiązania ogólnodostępne w sklepie Play, wykorzystujące sztuczną inteligencję w celu identyfikacji roślin, a także identyfikacji trapiących ich chorób. Są one przykładem praktycznego zastosowania technologii klasyfikacji w codziennym życiu.

2.4.1. Planta: Plant & Garden Care

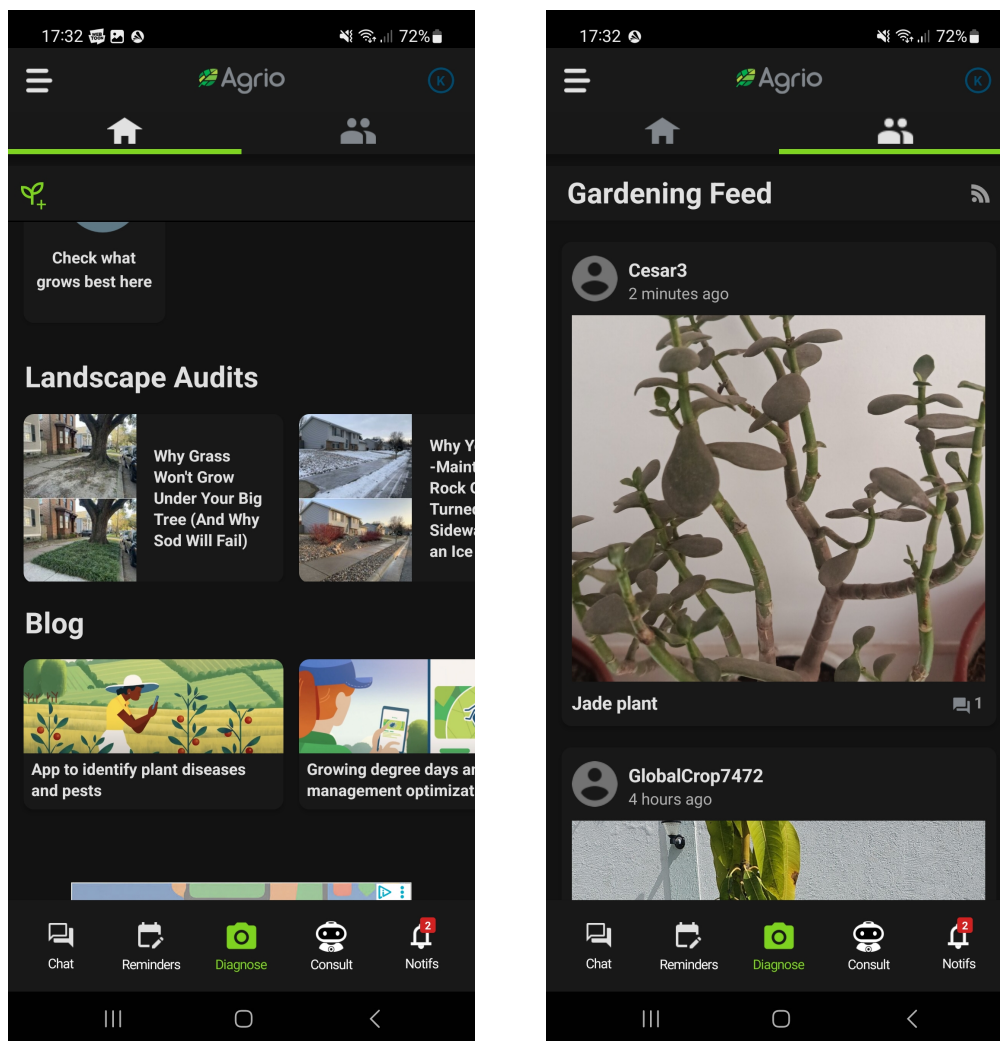
Aplikacja skierowana do osób zajmujących się hodowlą roślin. Aplikacji zawiera atlas roślin, w którym znajdują się informacje na temat trudności w hodowli, cech rośliny oraz szeroki zakres informacji na temat hodowli każdego gatunku z bazy. W ramach zakupu subskrypcji aplikacja umożliwia także identyfikację roślin oraz dręczących ich chorób ze zdjęcia oraz planowanie i przypominanie na temat różnych czynności dotyczących opieki nad rośliną np. podlewanie czy nawożenie[9].



Rysunek 2.4. Przykładowe zrzuty ekranu z aplikacji Planta[9].

2.4.2. Agrio - Plant diagnosis app

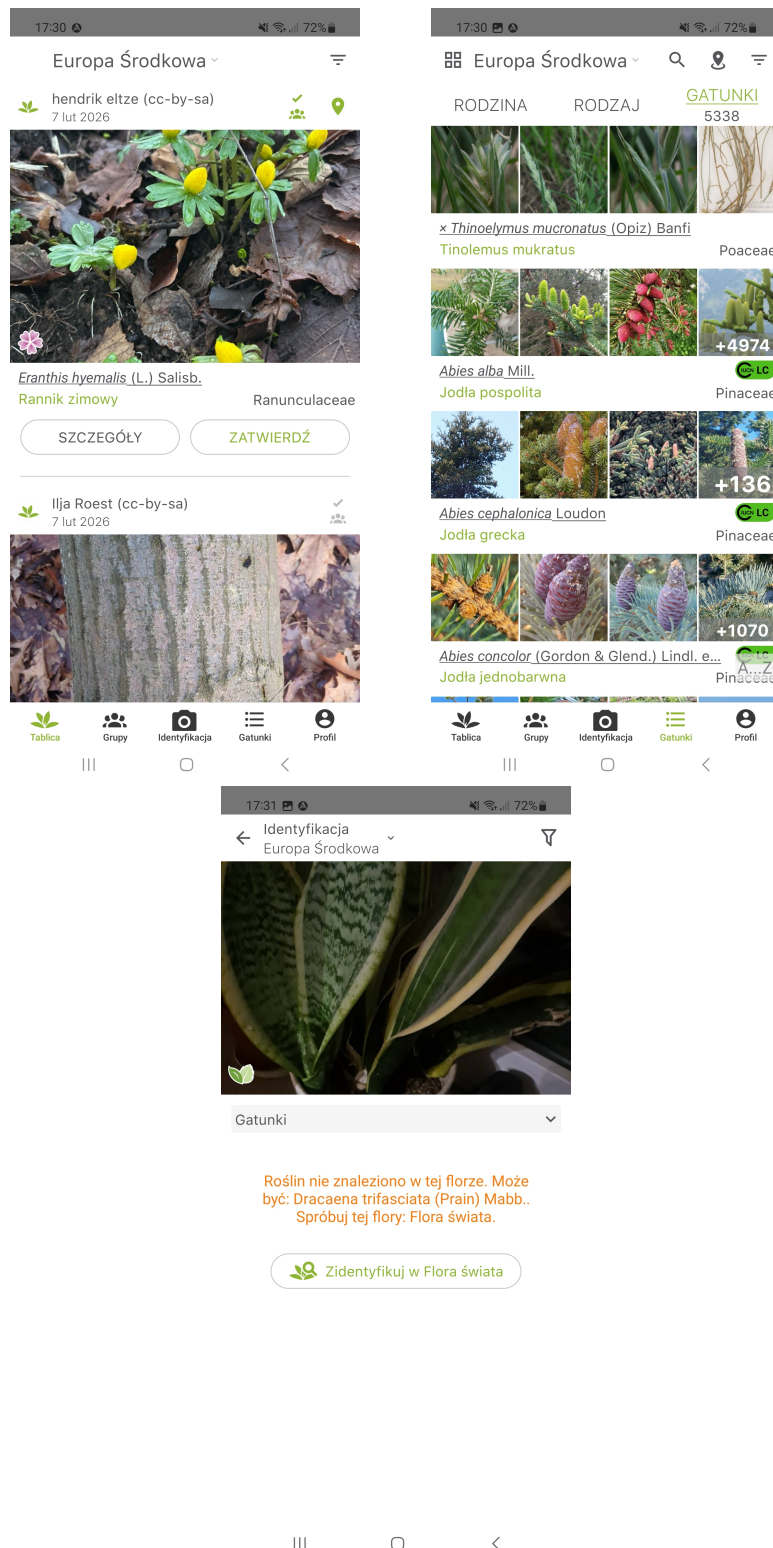
Aplikacja skierowana do miłośników roślin. Pozwala na wykonanie serii zdjęć chorej rośliny, przekazanie jej aplikacji, która następnie zadaje krótka serie pytań i na podstawie zdjęć oraz informacji z kwestionariusza, dokonuje diagnozy, jednak rozpoznaje tylko chorobę i rodzinę rośliny - nie podaje informacji szczegółowych na temat gatunku ze zdjęcia. Aplikacja pozwala także, za dodatkową opłatą, na skonsultowanie się z ekspertem. Prócz funkcji diagnozy, posiada także system chat SI, gdzie użytkownik może zadawać różne pytania na temat flory. Aplikacja posiada także funkcje społecznościowe[10].



Rysunek 2.5. Przykładowe zrzuty ekranu z aplikacji Agrio[10].

2.4.3. PlantNet Plant Identification

Aplikacja pozwala na identyfikację roślin na podstawie kilku metryk, np. na podstawie zdjęcia liści, kwiatu, czy ogólnego habitatu. Przedstawia także inne, prawdopodobne gatunki pasujące do dostarczonych danych. Posiada także atlas roślin, gdzie każdy gatunek z bazy ma przypisaną nazwę w wybranym z dostępnych języków, nazwę łacińską, zdjęcia, linki do źródeł informacji takich jak np. Wikipedia, oraz status gatunku w czerwonej księdze IUCN[11].



Rysunek 2.6. Przykładowe zrzuty ekranu z aplikacji[11].

3. Wprowadzenie do nauczania maszynowego

W poniższym rozdziale przedstawiono podstawy teoretyczne niezbędne do analizy wpływu liczby danych oraz ich jakości na wyniki klasyfikacji otrzymywane przez model. Omówiono podstawowe pojęcia wykorzystywane w dalszej części pracy, a następnie przedstawiono klasyczne metody klasyfikacji danych oraz nowoczesne architektury głębokich sieci neuronowych stosowanych w zadaniach klasyfikacji.

3.1. Podstawowe pojęcia

W poniższym podrozdziale przedstawiono wyjaśnienia wybranych pojęć, które są potem wykorzystywane w kolejnych rozdziałach pracy. Znajomość i zrozumienie tych pojęć jest niezbędna do poprawnej interpretacji opisanych w kolejnych rozdziałach metod, modeli oraz wyników przeprowadzonych badań.

3.1.1. Uczenie maszynowe

Dziedzina sztucznej inteligencji, zajmująca się algorytmami, które pozwalają komputerom na automatyczne uczenie się na podstawie danych.[12]

3.1.2. Rodzaje uczenia

- Uczenie nadzorowane - polega ono na dostarczeniu modelowi danych wejściowych z etykietami klas. Model tworzy funkcje celu, gdzie na wejściu otrzymuje dane, a na wyjściu etykietę. Podczas treningu model modyfikuje swoje wewnętrzne parametry tak, aby funkcja mogła dostarczonej mu dane przypisać odpowiednią etykietę.[13]
- Uczenie nienadzorowane - uczenie polegające na analizie zbioru danych zawierających wiele cech, a następnie nauki użytecznych właściwości struktury tego zbioru. Dane nie posiadają etykiet, w związku z tym model operuje jedynie na samej analizie cech. Algorytmy wykorzystujące tego typu uczenie polegają na podziale zbioru danych na klastry podobnych przykładów.
- Uczenie ze wzmocnieniem - uczenie polegające na przygotowaniu modeli środowiska, w którym uczy się wykonywać odpowiednie działania, które maksymalizują nagrodę. W przeciwieństwie do innych typów szkolenia, nie przygotowuje się zbioru, tylko środowisko. Model wchodzi z nim w interakcje i na podstawie poprzednich informacji oraz swojej polityki dokonuje akcji. Jego zadaniem jest udoskonalenie polityki w celu maksymalizacji nagrody otrzymywanej w wyniku podejmowanych akcji. [14]

3.1.3. Klasyfikacja

Analiza obrazów na podstawie ich treści w celu przyporządkowania ich do wcześniej zdefiniowanych klas.

3.1.4. Balans klas

Balans klas odnosi się do rozkładu liczby danych pomiędzy poszczególnymi klasami w ramach zbioru danych. Jeśli istnieją spore różnice w liczbie danych, między klasami, zbiór jest określany jako niezbalansowany, analogicznie, jeśli dane są równomiernie rozłożone to zbiór można określić jako zbalansowany.

3.1.5. Rodzaje zbiorów

Dane używane w ramach procesu treningu modelu oraz jego ewaluacji, dzielone są na trzy zbiory danych, z których każdy posiada unikalne zdjęcia oraz rolę w całym procesie uczenia:

- Zbiór treningowy (z ang. training set) - zbiór danych wykorzystywany przez model do treningu;
- Zbiór walidacyjny (z ang. validation set) - zbiór używany do wyznaczenia optymalnych hiperparametrów modelu;
- Zbiór testowy (z ang. test set) - zbiór danych wykorzystywanych do przetestowania wyszkolonego modelu;

3.1.6. Generalizacja

Zdolność modelu do klasyfikacji nie widzianych wcześniej danych, na podstawie przeprowadzonego treningu.[15]

3.1.7. Przeuczenie

Zjawisko występujące, kiedy model osiąga wysokie wyniki na zbiorze treningowym, jednak po otrzymaniu wcześniej niewidzianych danych jego skuteczność klasyfikacji spada. [16]

3.1.8. Regularyzacja

Zbiór technik matematycznych, których celem jest zmniejszenie błędu generalizacji, ale nie błędu uczenia. [15]

3.1.9. Uczenie transferowe

Uczenie transferowe (z ang. transfer learning) to technika uczenia sieci neuronowych, gdzie model trenowany na jednym zbiorze danych, wykorzystywany jest ponownie w podobnym zadaniu, szczególnie kiedy opiera się ono o ograniczone dane.

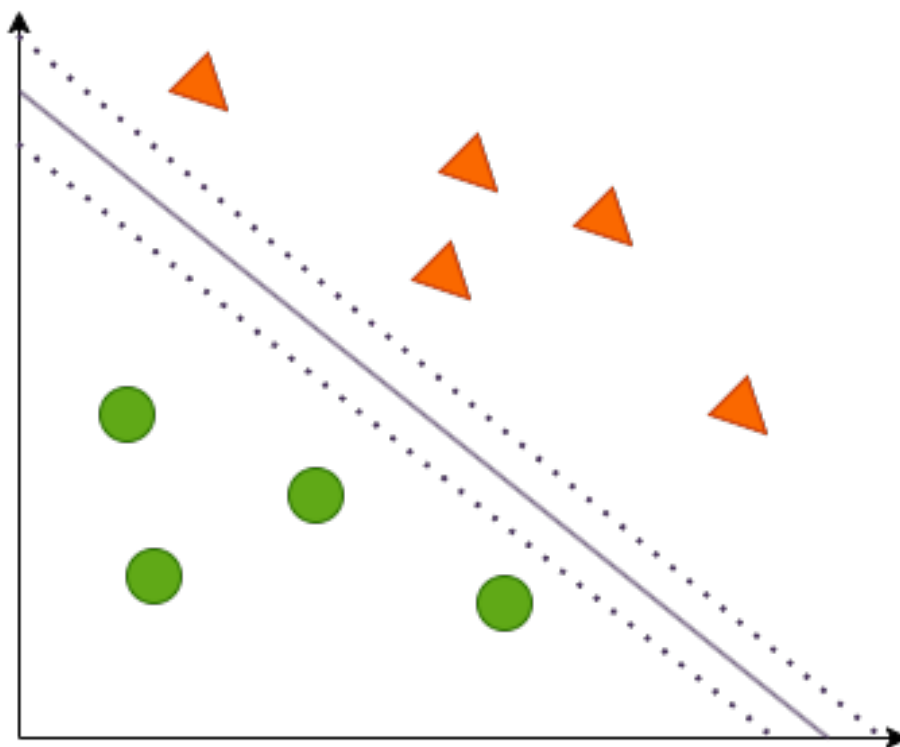
3.1.10. Dostrajanie modelu

Technika uczenia transferowego, polegająca na wykorzystaniu wstępnie wytrenowanego modelu, który jest dodatkowo trenowany na danych dotyczących konkretnego zadania, aby dostosować jego wagi do pożądanego zadania.

3.2. Klasyczne metody uczenia maszynowego

3.2.1. Maszyna wektorów nośnych - SVM

Maszyna wektorów nośnych (z ang. SVM - Support Vector Machine) to metoda uczenia nadzorowanego, najczęściej wykorzystywana w zadaniach klasyfikacji oraz regresji. Algorytm wyznacza najlepszą granicę dyskryminacji, bądź hiperpowierzchnię w przypadku danych wielowymiarowych, która pozwoli na odseparowanie różnych klas w zestawie danych. Sieci te dedykowane są do zadań klasyfikacji, gdzie jedną klasę separujemy możliwie dużym marginesem od pozostałych klas.



Rysunek 3.1. Schemat ilustrujący działanie algorytmu SVM

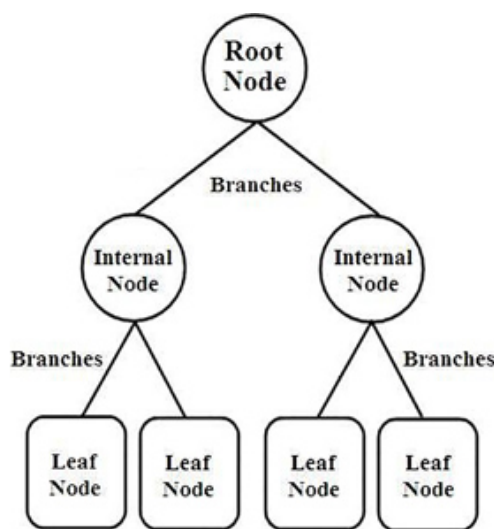
Algorytm SVM jest szczególnie wrażliwy na występowanie wyjątków. W niektórych zadaniach poprawna klasyfikacja jest przez to niemożliwa. W związku z tym, zamiast twardego marginesu, czyli optymalnego marginesu niedopuszczającego żadnych błędów, należy zastosować miękki margines - pozwala on na pewne błędy klasyfikacji wynikające z występowania wyjątków i umożliwia kompromis między szerokością marginesu, a liczbą błędów. W związku ze sposobem działania tego algorytmu nie nadaje się on do zadania klasyfikacji wieloklasowej, z małym wyjątkiem - separowanie jednej klasy od innych, jednak nie jest to przedmiot badania w tej pracy.[17][18]

3.2.2. Regresja liniowa

Regresja liniowa (z ang. Linear Regression) to algorytm uczenia maszynowego nadzorowanego służący do przewidywania wartości liczbowych. Modeluje zależność między zmienną zależną a jedną lub wieloma niezależnymi zmiennymi, przy założeniu, że jest ona liniowa. Regresja liniowa znajduje najlepiej dopasowaną prostą, która reprezentuje relację między wartościami rzeczywistymi, a przewidywanymi. Model działa dobrze, gdy występuje liniowa zależność. W momencie kiedy ta zależność jest nieliniowa, model może nie działać poprawnie. Dodatkowo, wartości odstające, czyli odbiegające od reszty zbioru, mogą znacząco wpłynąć na jakość predykcji. Jest to jednak prosty w zrozumieniu oraz implementacji algorytm nauczania maszynowego. Wykorzystywany jest on głównie w operacjach na danych liczbowych, np. prognozach finansowych, analizach danych czy marketingu. Niestety, ze względu na to, że operuje on na wartościach ciągłych, a klasyfikacja wymaga konkretnych klas, nie nadaje się on do zadania klasyfikacji. [19]

3.2.3. Drzewa decyzyjne

Drzewa decyzyjne to jedna z podstawowych metod uczenia maszynowego, którego historia sięga XX wieku. Algorytmy oparte na drzewach decyzyjnych znajdują zastosowanie w różnych dziedzinach, na przykład w zadaniu klasyfikacji, regresji czy wyboru cech. Procedura uczenia drzew decyzyjnych polega na rekurencyjnym dzieleniu danych na podzbiory co prowadzi do powstania struktury przypominającej drzewo, tak jak na rysunku 3.2.



Rysunek 3.2. Schemat budowy drzewa decyzyjnego [20]

Węzeł główny to początkowy punkt drzewa, reprezentuje on cały zbiór danych. Algorytm następnie identyfikuje cechę oraz progi, które prowadzą do najlepszego podziału na podstawie wybranego kryterium. Proces ten jest kontynuowany, aż zostanie osiągnięty warunek końcowy - może to być ustalona głębokość drzewa, bądź gdy wszystkie dane

w węzle należą do jednej klasy. Węzły końcowe, w których drzewo się kończy nazywane są liśćmi - reprezentują one wyniki bądź etykiety klas. Dzięki charakterystycznej budowie są one łatwe w zrozumieniu kolejności podejmowania decyzji. Efektywność drzew decyzyjnych jest jednak bardzo zależna od liczby oraz jakości danych, a przy dużej liczbie klas drzewo podatne jest na przeuczenie.[20]

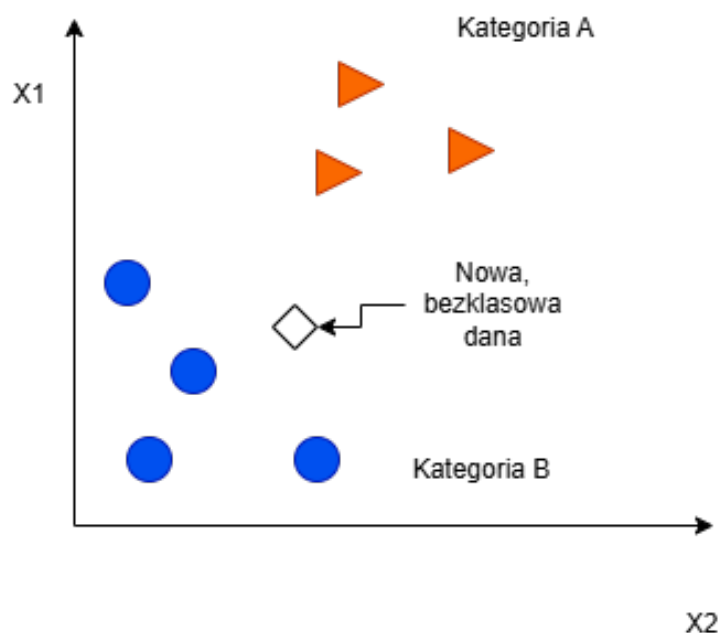
3.2.4. Algorytm najbliższych sąsiadów - k-NN

Algorytm najbliższych sąsiadów (k-NN) jest jednym z najprostszych algorytmów uczenia maszynowego bazującym na uczeniu nadzorowanym.[21] Znajduje zastosowanie głównie w zadaniu klasyfikacji. Jego działanie polega na wskazaniu do jakiej klasy należy nowy punkt w obszarze danych oznaczonych etykietami, np. klasa A i klasa B. Działanie algorytmu polega na konkretnych krokach:

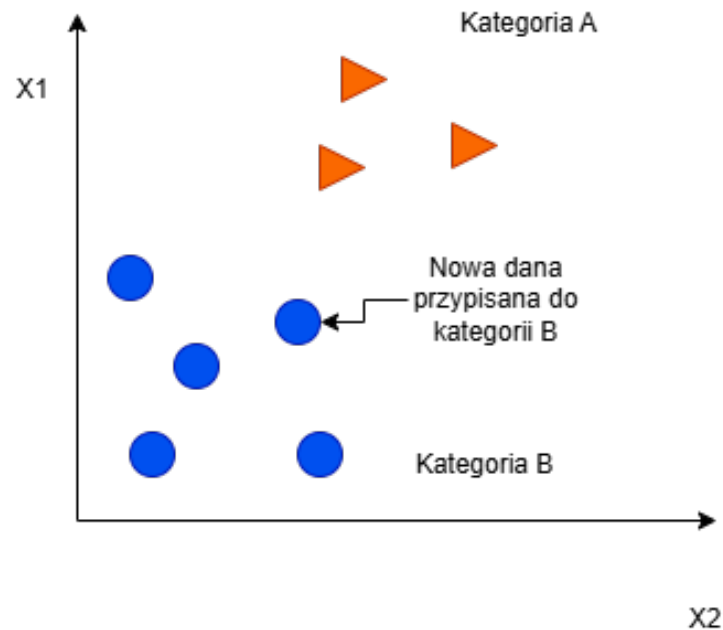
1. Wybierz liczbę sąsiadów k
2. Oblicz odległość do wszystkich sąsiadów
3. Spośród sąsiadów wybierz k najbliższych na podstawie wyliczonych odległości
4. Wśród wybranych sąsiadów k , policz ilość punktów należących do każdej kategorii
5. Do badanego punktu zostanie przypisana ta kategoria, która ma najwięcej klas w grupie najbliższych sąsiadów.

Odległość możemy wyliczyć stosując wybrany wcześniej wzór, na przykład, wzór na odległość euklidesową:

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



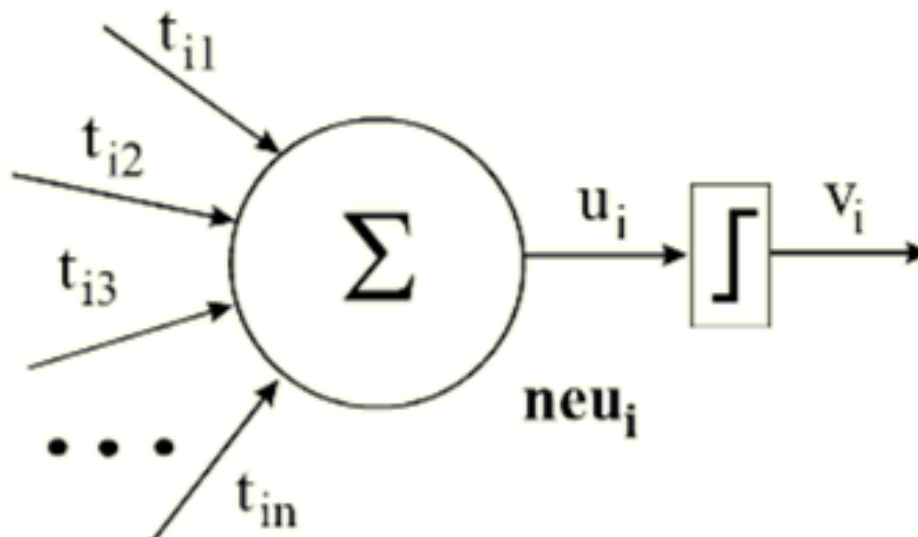
Rysunek 3.3. Dane przed wykorzystaniem algorytmu k-NN



Rysunek 3.4. Dane po wykorzystaniu algorytmu k-NN

3.3. Sztuczne sieci neuronowe - ANN

Sieci neuronowe to techniki analityczne tworzone na wzór ludzkiego mózgu, posiadająca zdolność do przewidywania nowych obserwacji na podstawie wcześniejszych obserwacji, po przeprowadzeniu procesu uczenia opartego o istniejące dane. Pierwszym krokiem jest zaprojektowanie architektury sieci. Zawierać ona powinna określoną liczbę warstw, z których każda posiada określoną liczbę neuronów. Neuron jest matematycznym modelem, którego zadaniem jest odbieranie sygnałów otrzymywanych od innych neuronów, przetwarzanie sumarycznego sygnału przy zastosowaniu funkcji aktywacji, a następnie wysłanie przetworzonej próbki do innych neuronów znajdujących się w sieci. Wielkość oraz struktura sieci zależy od badanego zjawiska. [22]

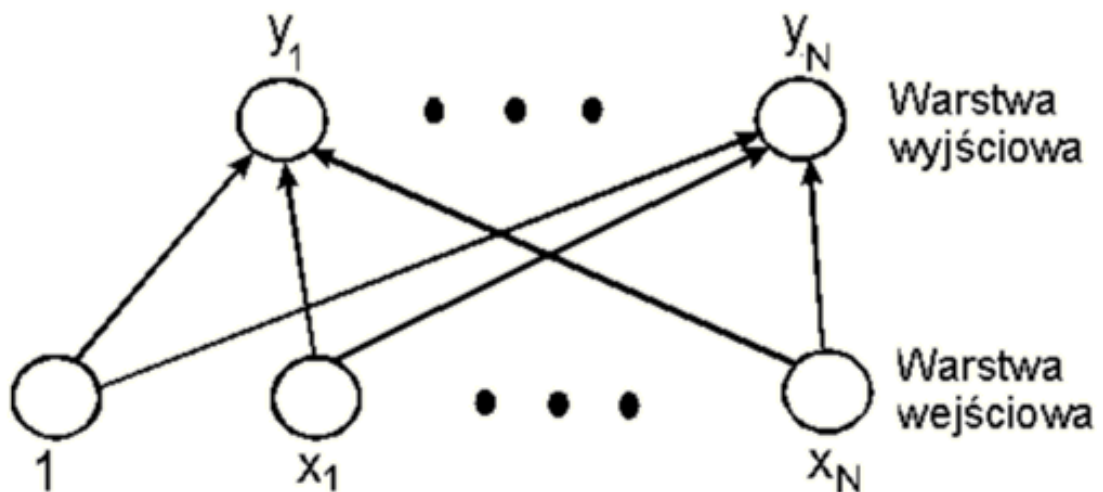


Rysunek 3.5. Schemat budowy neuronu[23]

W literaturze znajduje się wiele modeli neuronów, które różnią się między sobą sposobem sumowania otrzymanych danych oraz stosowaną funkcją aktywacji, która decyduje, w którym momencie wysyłany jest sygnał zwrotny i o jakiej mocy. Jednym z podstawowych modeli neuronu jest model McCullocha-Pittasa, który został zaproponowany w 1943 roku. Neurony w ramach sieci komunikują się ze sobą poprzez połączenia synaptyczne, których wielkość określa waga, oznaczoną na rysunku znakiem t_{ij} , który symbolizuje wagę połączenia neuronu j z neuronem i . Suma przemnożonych przez wagi sygnałów nazywana jest potencjałem wejściowym i oznaczona jest u_i . Potencjałem wyjściowym jest odpowiedź neuronu na zadany potencjał wejściowy, którego charakterystyka określana jest przez funkcje aktywacji. [23]

3.3.1. Perceptron

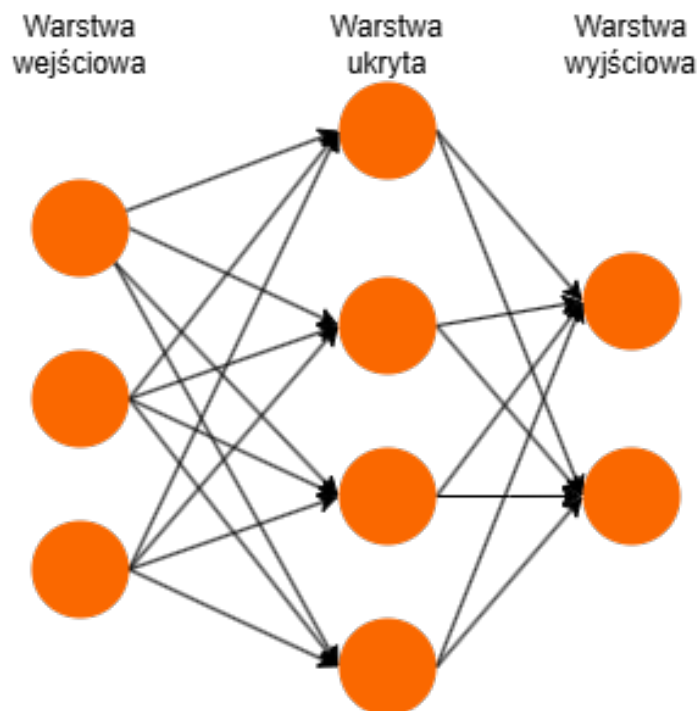
Jednym z podstawowych modeli sieci jednokierunkowych jest perceptron, którego schemat przedstawiony jest na rysunku 3.6. Jest on zbudowany z N neuronów wejściowych oraz M neuronów wyjściowych. Perceptron ma za zadanie odwzorowanie zadanego zbioru wektorów wejściowych na zadany zbiór wektorów wyjściowych. W związku z tym zbiór treningowy składa się z par wektorów. Nauka perceptronu polega na stopniowej modyfikacji wag łączących neurony wejściowe z wyjściowymi, tak żeby, po zakończeniu szkolenia, podanie wektora skutkowało wygenerowaniem odpowiadającego mu wektora wyjściowego.[23]



Rysunek 3.6. Schemat budowy perceptronu [23]

3.3.2. Perceptron wielowarstwowy - MLP

Perceptron wielowarstwowy - MLP (z ang. Multilayer Perceptron) to jeden z podstawowych typów sieci neuronowych. Sieci te muszą posiadać warstwę wejściową, która otrzymuje dane oraz warstwę wyjściową, która prezentuje wynik. Pomiędzy nimi, może znajdować się jedna, lub więcej warstw ukrytych.



Rysunek 3.7. Schemat ilustrujący budowę MLP.

W poszczególnych warstwach może występować różna liczba neuronów. W warstwie wejściowej odpowiada ona zwykle liczbie parametrów, które opisują dane wejściowe, a w warstwie wyjściowej np. liczba klas. Sieci te posiadają małą liczbę warstw ukrytych - duża liczba warstw ukrytych charakteryzuje sieci głębokie. [22][24] [23]

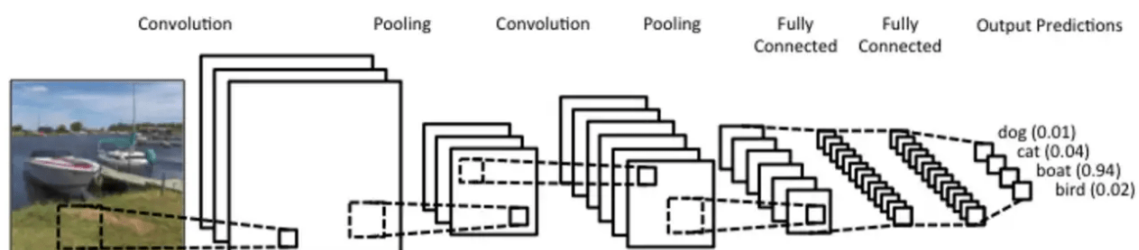
3.4. Uczenie głębokie - Deep learning

Uczenie głębokie to podzbiór uczenia maszynowego, oparty na sztucznych sieciach neuronowych, w których znajdziemy dużą liczbę warstw ukrytych. [25] Sieci głębokie mogą rozwiązywać bardziej złożone problemy, przy większej liczbie danych, niż proste architektury sieci neuronowych. Dzięki ukrytym warstwom mają one dużą złożoność obliczeniową, co wiąże się też z większą liczbą zasobów oraz czasu potrzebnego do przeprowadzenia treningu.

3.4.1. Konwolucyjne sieci neuronowe - CNN

Konwolucyjne sieci neuronowe to typ architektury uczenia głębokiego, opierającej się na czterech kluczowych conceptach: warstw konwolucyjnych, warstw łączących, funkcji aktywacji oraz w pełni połączonych warstw. Modele te znajdują zastosowanie w klasyfikacji obrazów, detekcji obiektów, rozpoznawaniu mowy, analizie danych oraz więcej.

W przypadku analizy obrazów model przyjmuje dane wejściowe w postaci macierzy, bądź tensora.



Rysunek 3.8. Schemat budowy CNN.[26]

Warstwa konwolucyjna (z ang. convolutional layer) stanowi najważniejszy element architektury modelu. W warstwach model przetwarza otrzymane dane, aby wykryć lokalne cechy takie krawędzie czy tekstury. Warstwa posiada jądro (z ang. kernel), które przesuwa się po obrazie, mnoży skalarne wartości filtru i pikseli tworząc mapę cech (z ang. feature map), która jest nową reprezentacją obrazu. Zawiera ona informacje wydobyte przez jądro. Jądro, które nazywane też jest czasem filtrem, przesuwane jest o z góry narzucony krok.

Warstwa łącząca (z ang. pooling layer) to warstwa zmniejszająca rozdzielczość map cech, zachowując najważniejsze informacje. Pozwala to na zmniejszenie liczby obliczeń oraz zapobiegnięcie przeuczenia. Warstwa ta najpierw zmniejsza rozdzielczość mapy cech,

wybierając z okna najważniejszą wartość i zapisuje ją w nowej mapie. Następnie przesuwa się o stały krok. Wartości są wybierane w różny sposób - dla np. warstwy Max Pooling, wybierana jest największa wartość w oknie, a dla warstwy Average Pooling wyliczana jest średnia wartości w oknie i to ona jest zapisywana w nowej mapie.

Funkcja aktywacyjna (z ang. activation function) jest aplikowana do wyliczonej sumy danych wejściowych przez warstwę konwolucyjną. Wprowadza ona nieliniowość do modelu. Pozwala to modelowi na naukę skomplikowanych wzorców i zależności. Istnieje wiele funkcji aktywacji, każda z nich stosowana jest w innym celu. W przypadku klasyfikacji binarnej stosuje się funkcję sigmoidalną, a w przypadku klasyfikacji wieloklasowej zastosować można funkcje ReLu, bądź Softmax.

Warstwa w pełni połączona warstwa to warstwa, w której wszystkie neurony są połączone ze wszystkimi neuronami z warstwy poprzedniej. W CNN są to warstwy odpowiadające za generowanie predykcji na podstawie cech wyuczonych przez poprzednie warstwy.[27]

3.4.2. Transformer

Transformer to architektura uczenia głębokiego skupiająca się na modelowaniu relacji między danymi. W przeciwieństwie do CNN, przetwarza ona całe sekwencje danych jednocześnie, zamiast krok po kroku. Z tego powodu znajdują one zastosowanie w przetwarzaniu języka naturalnego czy analizie obrazów. Transformery opierają się na kilku kluczowych conceptach: mechanizm samo-uwagi, wielogłowej uwagi, kodowania pozycyjnego oraz wektorów reprezentacji.

Mechanizm samo-uwagi(z ang. self-attention mechanism) przetwarza np. wszystkie słowa w zdaniu, czy cały obraz jednocześnie i identyfikuje, które fragmenty są ważne i jakie są relacje między nimi, aby móc określić kontekst i znaczenie stojące za całością. Każdy fragment w sekwencji jest mapowany na trzy wektory: Query (Q), Key(K) i Value(V). Wyliczana jest następnie wartość uwagi według wzoru:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_K}})V$$

Wielogłowa uwaga(z ang. multi-head attention) to mechanizm pozwalający modelowi na jednoczesne analizowanie różnych zależności występujących w danych takich jak kolor, tekstura czy kształt. Zamiast pojedynczego mechanizmu uwagi stosuje się kilka niezależnych głów jednocześnie, z których każda koncentruje się na innych zależnościach między elementami sekwencji. Wyniki ze wszystkich głów są następnie łączone i poddawane projekcji liniowej, aby stworzyć końcowe wyjście mechanizmu uwagi. Całość pozwala modelowi uchwycić więcej powiązań i wzorców, niż przy użyciu pojedynczej głowy.

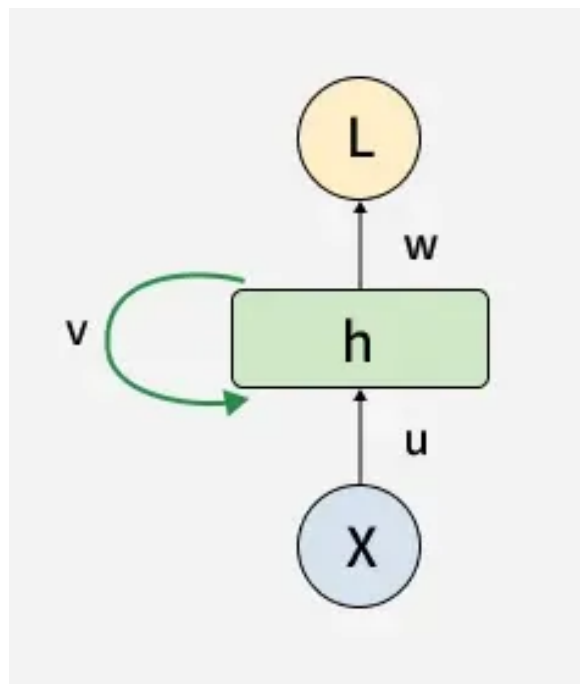
Transformery nie mogą pracować bezpośrednio na surowych danych, ponieważ wymagają numerycznych reprezentacji. Z tego powodu, każdy token, który jest fragmentem danych, np. fragment słowa, bądź słowo, jest przekształcane w wektor, który nazywany jest

wektorem reprezentacyjnym. Zawierają one informacje na temat znaczenia, bądź relacji między elementami danych, gdzie fragmenty podobne do siebie znajdują się blisko siebie, a te różniące dalej.

Z powodu procesowania danych równolegle, transformery nie są w stanie określić kolejności danych. Aby rozwiązać ten problem, do wektorów reprezentacyjnych, dodawane są kodowania pozycyjne, które dostarczają modelowi informacje o pozycji każdego tokenu w zbiorze danych.[28]

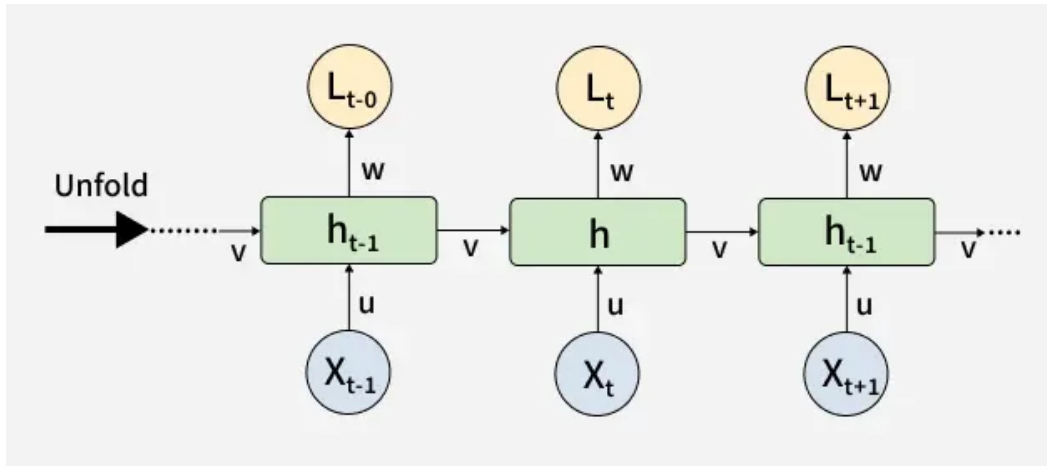
3.4.3. Rekurencyjne sieci neuronowe - RNN

Rekurencyjne sieci neuronowe to typ architektury sieci neuronowych przeznaczonej do detekcji wzorców w sekwencji danych. Przykładami takich danych są tekst, pismo odręczne, genomy lub numeryczne szeregi czasowe. Sieci tego typu mogą zostać wykorzystane również do analizy obrazów, jeśli dane zostaną odpowiednio wcześniej podzielone na fragmenty i potraktowane jako dane sekwencyjne. Podstawową jednostką przetwarzającą w RNN jest jednostka rekurencyjna, która przechowuje ukryty stan (z ang. hidden state) - zawiera on informacje o poprzednich elementach sekwencji.



Rysunek 3.9. Przedstawienie jednostki rekurencyjnej.[13]

Poprzez przekazywanie stanu ukrytego pomiędzy kolejnymi krokami, jednostki rekurencyjne mogą zapamiętać informacje z wcześniejszych etapów, które mogą wpływać na wynik generowany przez sieć w kolejnym kroku. Aby umożliwić sieciom RNN uczenie się zależności występujących w danych sekwencyjnych, stosuje się concept rozwijania sieci, polegający na przedstawieniu struktury rekurencyjnej jako szeregu kolejnych kroków czasowych.



Rysunek 3.10. Przedstawienie rozwijania RNN

Pozwala to na zastosowanie propagacji wstecznej przez czas - BPTT (z ang. Backpropagation Through Time). Jest to sposób propagacji błędów i obliczania gradientów w sieciach RNN. Ze względu na architekturę RNN, BPTT uwzględnia wszystkie stany przez kolejne kroki czasowe sekwencji. Umożliwia to aktualizację wag z uwzględnieniem wszystkich zależności między elementami danych sekwencyjnych. Rekurencyjne sieci neuronowe znajdują zastosowanie w dziedzinach operujących na danych sekwencyjnych - takich jak np. prognoza pogody, czy generacja muzyki. Związku z tym, że modele RNN nie są powszechnie stosowane w zadaniach klasyfikacji obrazów, nie zostały one uwzględnione w ramach badania.[29]

3.5. Metryki ewaluacji wyników

Ocena skuteczności modeli klasyfikacji obrazów wymaga zastosowania odpowiednio dobranych metryk ewaluacyjnych, umożliwiających obiektywną ocenę jakości uzyskiwanych predykcji. Odpowiedni dobór metryk pozwala na wieloaspektową interpretację otrzymanych wyników, także przy obecności niezbalansowanych danych. Umożliwiają one ogólną ocenę modelu, jak i analizę rodzaju popełnianych przez niego błędów. Poniżej przedstawiono wybrane metryki ewaluacyjne wraz ze wzorami oraz krótkim opisem użycia. Metryki te są powszechnie stosowane w badaniach dotyczących klasyfikacji obrazów.

3.5.1. Macierz pomyłek

Macierz pomyłek (z ang. confusion matrix) jest podstawowym narzędziem służącym do analizy wyników klasyfikacji. Stanowi ona podstawę do wyznaczania bardziej złożonych metryk, takich jak precyzja czy czułość. Jest to tabela krzyżowa pozwalająca na porównanie wyników predykcji do oryginalnego przypisania klas do danych. Dzieli ona wynik klasyfikacji na cztery kategorie: TP (true positive), TN (true negative), FP (false positive) i FN (false negative) [30]. Dla klasyfikacji wieloklasowej, macierz pomyłek, jest

macierzą o wymiarach $n \times n$, gdzie n to liczba klas, co przedstawiono w przykładzie dla klasy 'B' poniżej.

		Klasy przewidziane przez model			
		Klasy	A	B	C
Prawdziwa klasyfikacja	A	TN	FP	TN	
	B	FN	TP	FN	
	C	TN	FP	TN	

Rysunek 3.11. Przykładowa macierz pomyłek dla trzech klas, z wynikami otrzymywanymi dla klasy 'B'.

3.5.2. Dokładność

Wzór na Dokładność (z ang. Accuracy) określa stosunek liczby poprawnie przypisanych przez model klas do liczby wszystkich dokonanych klasyfikacji. [30]

$$Accuracy = \frac{TP + TN}{TP + TN + FN + FP}$$

Wartość tej metryki określa odsetek poprawnie dokonanych predykcji przez model. Jest to podstawowa metryka, pozwalająca na szybką ocenę skuteczności modeli, zwracająca skuteczność predykcji modelu na całym zbiorze danych, nie bierze jednak pod uwagę niezbalansowania zbioru danych. Ze względu na prostą interpretację metryka ta jest użyteczna do szybkiej i wstępnej oceny modelu.

3.5.3. Precyzja

Precyzja (z ang. Precision) to miara informująca nas, jaki odsetek przypisań danej klasy dokonanych przez model, zgadza się z oryginalnymi przypisaniami. Innymi słowy, z jak często model dokonuje poprawnej predykcji danej klasy. [30]

$$Precision = \frac{TP}{TP + FP}$$

Miara ta jest używana dla konkretnych klas, jednak jej średnia, może zostać użyta do oceny całego modelu. Obliczanie tej metryki osobno dla każdej klasy pozwala na ocenę, czy model nie wykazuje tendencji do częstszego przypisywania określonych klas.

3.5.4. Czułość

Czułość (z ang. recall) to miara informująca nas, jaki odsetek danych, które oryginalnie mają daną klasę, zostało poprawnie sklasyfikowanych przez model. Jest więc to liczba poprawnych przypisań danej klasy podzielona przez liczbę danych należących do tej klasy w zbiorze. [30]

$$Recall = \frac{TP}{TP + FN}$$

Miara ta pozwala ocenić, w jakim stopniu model jest w stanie poprawnie przypisać klasę do danych. Możliwe jest też użycie średniej tej metryki, co pozwala na uzyskanie oceny całego modelu, jednak uniemożliwia to analizę wyników klasyfikacji poszczególnych klas.

3.5.5. Miara F1

Miara F1 (z ang. F1-Score) jest próbą znalezienia pojedynczej metryki, pozwalającej na jak najbardziej obiektywną ocenę modelu. Metryka agreguje metryki Precyzji i Czułości z wykorzystaniem średniej harmonicznej. [30]

$$F1 - Score = 2 * \left(\frac{precision * recall}{precision + recall} \right)$$

Przedstawiony wyżej wzór jest używany tylko w ramach klasyfikacji binarnej. Aby użyć tej miary w ramach klasyfikacji wieloklasowej należy, obliczyć odpowiednie średnie wcześniej wykorzystanych miar. W ramach tego otrzymujemy dwie nowe, ogólne miary: Macro F1-Score, które wykorzystuje arytmetyczną średnią Precyzji i Czułości, oraz Micro F1-Score – wykorzystująca średnie harmoniczną.[30]

$$MacroF1 - Score = 2 * \left(\frac{ArithmeticPrecision * ArithmeticRecall}{ArithmeticPrecision + ArithmeticRecall} \right)$$

$$MicroF1 - Score = 2 * \left(\frac{HarmonicPrecision * HarmonicRecall}{HarmonicPrecision + HarmonicRecall} \right)$$

4. Modele

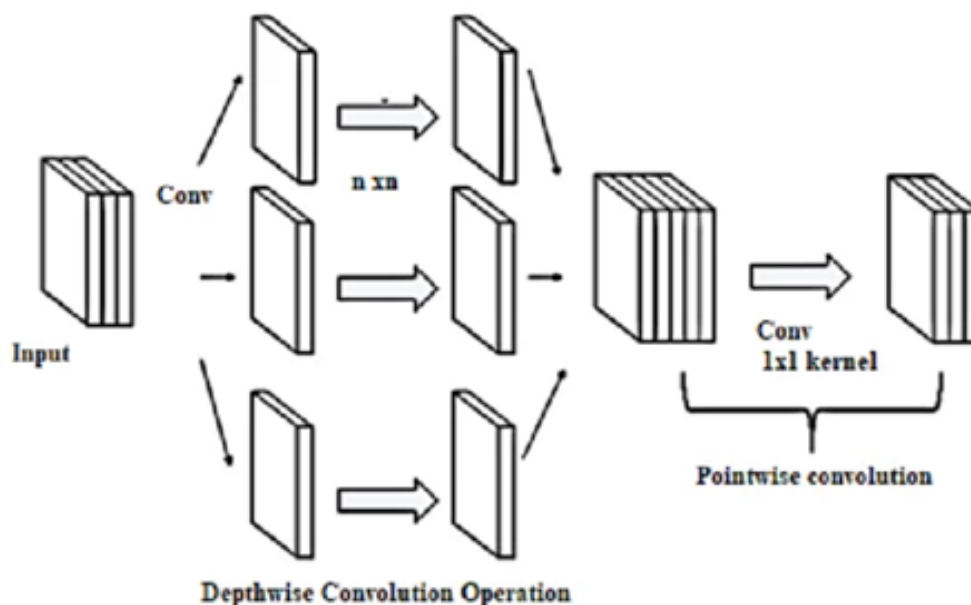
W poniższym rozdziale przedstawiono rodziny modeli głębokiego uczenia wykorzystane w pracy w ramach badania ich możliwości na wybranym wcześniej zbiorze danych zawierającym rośliny uprawne i choroby. Cztery z tych rodzin modeli reprezentują klasyczne podejście w postaci architektury CNN – ResNet, MobileNet, EfficientNet i DenseNet oraz jedna rodzina modeli będący implementacją transformera wizyjnego – ViT. Wybór modeli podyktowany był próbą rozszerzenia badań wcześniej przedstawionych w przeglądzie literatury poprzez uwzględnienie modeli, które nie były analizowane, bądź bezpośrednio ze sobą porównywane. Uwzględnienie zarówno szeroko stosowanych modeli CNN, jak i nowszej architektury ViT pozwala na porównanie podejść rozwijanych w różnych okresach rozwoju głębokiego uczenia. Każdy z poniższych podrozdziałów przedstawia jedną z rodzin oraz przedstawia zasadę działania danej rodziny modeli oraz jej charakterystyczne cechy.

4.1. MobileNet

MobileNet to rodzina lekkich modeli konwolucyjnych sieci neuronowych (CNN). Została ona opracowana przez firmę Google, zaprojektowana z myślą o wykonywaniu zadań z dziedziny wizji komputerowej - klasyfikacja obrazów, czy detekcja obiektów. Są to małe modele o niskiej liczbie parametrów i małych opóźnieniach, zaprojektowane z myślą o ograniczeniach zasobów obliczeniowych, występujących w wielu urządzeniach elektronicznych, głównie telefonach mobilnych.

Charakterystyczną cechą tej rodziny modeli jest zastosowanie splotów separowanych głębinowo (z ang. depthwise separable convolutions), które znacząco zmniejszają koszt obliczeniowy w porównaniu ze standardowymi warstwami konwolucyjnymi. Technika ta polega na rozdzieleniu standardowej konwolucji na konwolucję głębościową (z ang. depthwise convolution) oraz konwolucję 1×1 , nazywaną konwolucją punktową (z ang. pointwise convolution).

Konwolucja głębościowa wykorzystuje pojedynczy filtr dla każdego kanału wejściowego. Następnie konwolucja punktowa stosuje konwolucję 1×1 , aby połączyć wyniki uzyskane w ramach konwolucji głębościowej.



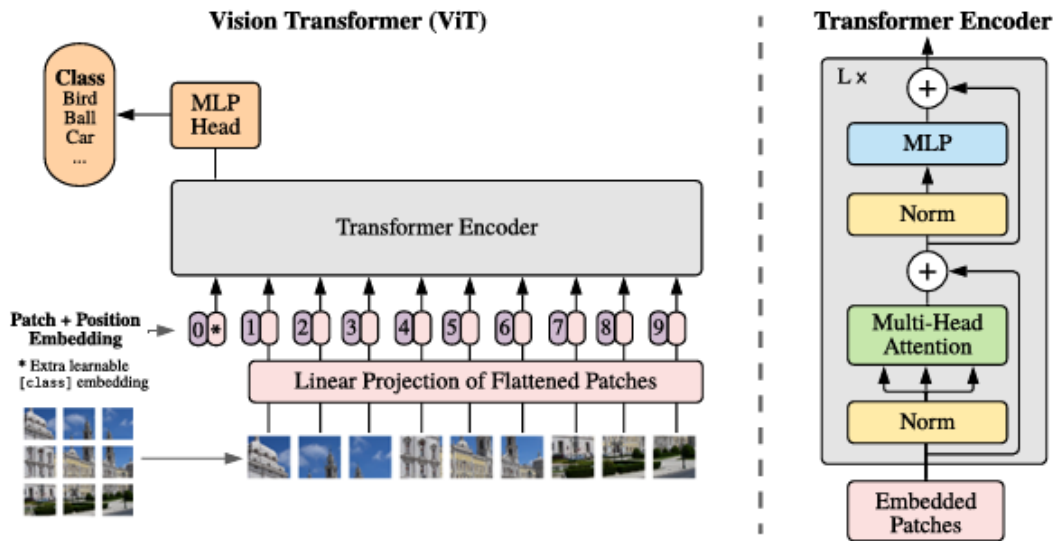
Rysunek 4.1. Przedstawienie działania konwolucji głębokościowej.[31]

Różnica między standardową konwolucją a techniką zastosowaną w tej rodzinie modeli jest znacząca. Standardowa konwolucja jednocześnie wykonuje filtrowanie oraz łączenie informacji w nowe dane wyjściowe w ramach jednego kroku, natomiast zastosowanie splotów separowanych głębiniowo rozdziela to na dwie warstwy, jedna odpowiadająca za filtrowanie oraz drugą, oddzielną, odpowiadającą za łączenie. Pozwala to na znaczne zmniejszenie liczby obliczeń oraz wielkości modelu, przy jednoczesnym zachowaniu zdolności sieci do ekstrakcji istotnych cech obrazu. [32]

4.2. Vision transformer - ViT

Vision Transformer (ViT, z ang. transformer wizyjny) to architektura sieci neuronowej przeznaczona do analizy obrazów. Działa ona odmiennie od modeli konwulucyjnych sieci neuronowych. Architektura ta opiera się na transformerze oraz mechanizmie samo-uwagi (z ang. self-attention), pozwalający modelowi analizować zależności między różnymi fragmentami obrazu. Umożliwia to modelowanie zarówno lokalnych, jak i globalnych zależności między elementami obrazu.

ViT traktuje obraz jako sekwencję fragmentów (z ang. patch). Dzięki temu możliwe jest wykorzystanie mechanizmu samo-uwagi do modelowania zależności między różnymi fragmentami znajdującymi się w dużej i małej odległości od siebie.



Rysunek 4.2. Przedstawienie budowy oraz działania ViT.[33]

Standardowy transformer otrzymuje dane wejściowe w postaci jednowymiarowej sekwencji tokenów. Aby model mógł korzystać z obrazów dwumiarowych jako danych wejściowych, obraz jest przekształcany w sekwencje spłaszczonych, dwuwymiarowych fragmentów obrazu, gdzie (H, W) oznacza rozdzielczość obrazu wejściowego, C liczbę jego kanałów, a (P, P) określa rozmiar pojedynczego fragmentu. Zmienna P jest narzucona z góry, określona w nazwie modelu. Zatem liczba uzyskanych fragmentów, a jednocześnie długość sekwencji wejściowej dla transformera, wynosi:

$$N = \frac{HW}{P^2}$$

Transformer wykorzystuje stały wymiar wektorów reprezentacji w swoich warstwach, zatem poszczególne fragmenty obrazu są spłaszczane, a następnie przekształcane do jednowymiarowej przestrzeni. Wynik tej projekcji stanowi reprezentację fragmentu obrazu (z ang. patch embedding)

Na początku sekwencji reprezentacji fragmentów obrazu umieszczany jest nauczalny wektor reprezentacji (z ang. learnable embedding). Wektor ten służy do reprezentowania informacji zawartych w całej sekwencji.

Do reprezentacji fragmentów obrazu, dodawane są kodowania pozycyjne (z ang. position embeddings). Przechowują one informacje o położeniu konkretnych fragmentów.

Tak stworzony wektor jest używany jako dana wejściowa do enkodera. Dzięki takiej konstrukcji, model ten jest w stanie tworzyć globalne powiązania między fragmentami obrazu.[33]

4.3. ResNet

ResNet to rodzina modeli konwolucyjnych sieci neuronowych (CNN), stworzona przez firmę Microsoft w 2015 roku. Głównym celem opracowania tej architektury było rozwiązanie problemów pojawiających się podczas trenowania bardzo głębokich sieci neuronowych. Wraz ze zwiększaniem liczby warstw sieci, model powinien w teorii być zdalny do uczenia się coraz bardziej złożonych reprezentacji danych. W praktyce jednak dalsze zwiększanie głębokości modelu prowadzić może do pogorszenia jego wyników. ResNet rozwiązuje ten problem poprzez implementację połączeń rezydualnych (z ang. residual connection).

W klasycznej sieci konwolucyjnej każda warstwa otrzymuje dane wyjściowe z warstwy poprzedzającej i przekształca je w kolejną reprezentację cech. W przypadku bardzo głębokich sieci neuronowych może to utrudniać proces treningu. Podczas propagacji wstecznej wartości gradientu przekazywane do wcześniejszych warstw sieci stają się coraz mniejsze, co w konsekwencji powoduje, że parametry tych warstw są aktualizowane w niewielkim stopniu. Nazywane jest to problemem zanikającego gradientu (z ang. vanishing gradient).

Architektura ResNet rozwiązuje ten problem poprzez wprowadzenie bezpośrednich połączeń, nazywanych połączeniami skrótowymi (z ang. shortcut connections, lub skip connections). Zamiast wymagać od kolejnych warstw nauczenia się bezpośrednio funkcji odwzorowania, warstwy uczą się funkcji rezydualnej, tak aby:

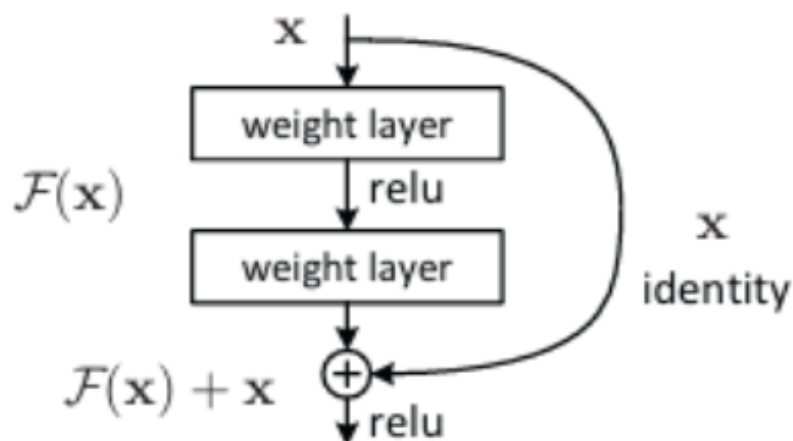
$$F(x) = H(x) - x$$

gdzie x oznacza dane wejściowe, $F(x)$ transformację dokonywaną przez warstwy konwolucyjne, a $H(x)$ oczekiwana funkcją odwzorowania. Całość jest wykonywana w ramach jednego bloku rezydualnego. W związku z tym funkcje odwzorowania można zapisać jako:

$$H(x) = F(x) + x$$

Ostatecznie zatem wyjście z takiego bloku powstaje poprzez dodanie danych wejściowych do transformacji dokonywanej w ramach bloku rezydualnego.

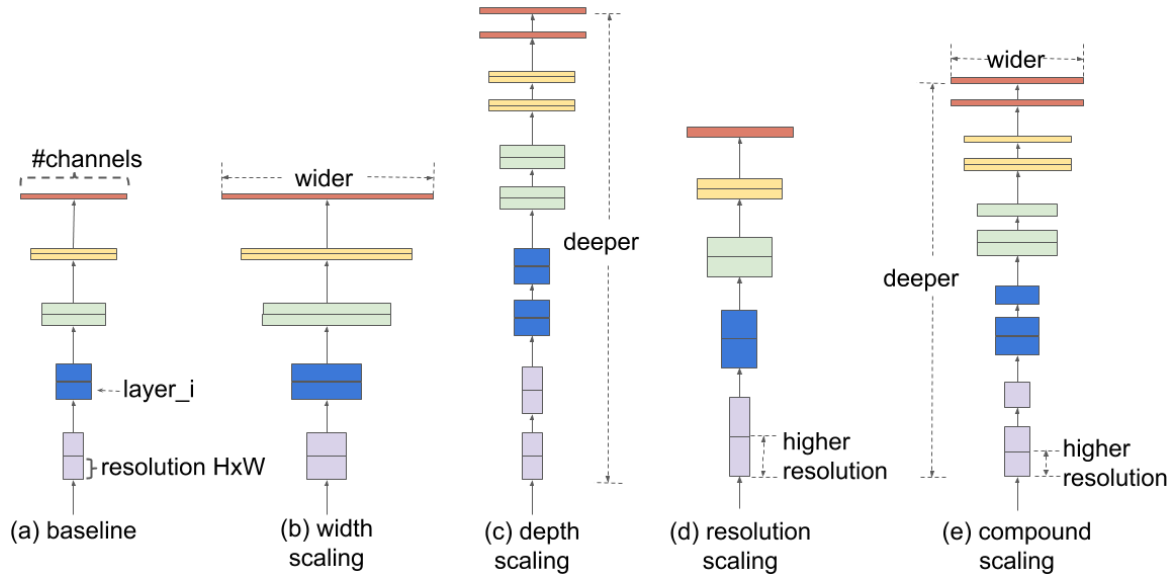
Rozwiązanie to pozwala sieci na naukę różnicy między oczekiwanym wyjściem a wejściem bloku, zamiast dokonywać aproksymacji docelowej. [25]



Rysunek 4.3. Przedstawienie budowy oraz działania bloku rezydualnego.[25]

4.4. EfficientNet

EfficientNet to rodzina modeli konwolucyjnych sieci neuronowych(CNN) stworzonych przez Google. Architektura została opracowana z myślą o uzyskaniu kompromisu między dokładnością modelu a jego kosztem obliczeniowym. W innych sieciach głębokich głębokość, szerokość oraz rozdzielczość były skalowane niezależnie od siebie. Naukowcy stojący za tą architekturą zauważyli, że zwiększenie rozmiaru sieci może mieć pozytywny efekt na dokładność uzyskiwanej klasyfikacji, jednak sposób skalowania modelu ma istotny wpływ na uzyskiwane rezultaty. Dla przykładu, dla obrazów o wyższej rozdzielczości, zwiększenie głębokości sieci pomaga polom recepcyjnym w uchwyceniu podobnych cech. Analogicznie, = zwiększenie szerokość sieci może umożliwić znajdowanie bardziej złożonych wzorców w obrazach o większej rozdzielczości. W ramach architektury EfficientNet zaproponowano metodę skalowania złożonego (z ang. compound scaling).



Rysunek 4.4. Skalowanie modelu (a) to bazowy przykład modeli; (b)-(d) są modelami, w których zwiększono tylko jeden wymiar; (e) to model w ramach, którego zastosowano skalowanie złożone.[34]

Skalowanie złożone wykorzystuje współczynnik złożony – do jednolitego i systematycznego skalowania szerokości, głębokości oraz rozdzielczości sieci:

$$\text{głębokość} : d = \alpha^\phi$$

$$\text{szerokość} : w = \beta^\phi$$

$$\text{rozdzielczość} : r = \gamma^\phi$$

przy warunkach:

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

$$\alpha \geq 1$$

$$\beta \geq 1$$

$$\gamma \geq 1$$

gdzie, α , β , i γ są stałymi wyznaczonymi przez autorów eksperymentalnie w ramach przeszukiwania siatki. ϕ jest określoną przez użytkownika współczynnikiem, który kontroluje ile zasobów obliczeniowych jest przeznaczonych na skalowanie modelu, a α , β , i γ określają jak te dodatkowe zasoby są rozdzielane między głębokość, szerokość i rozdzielczość.

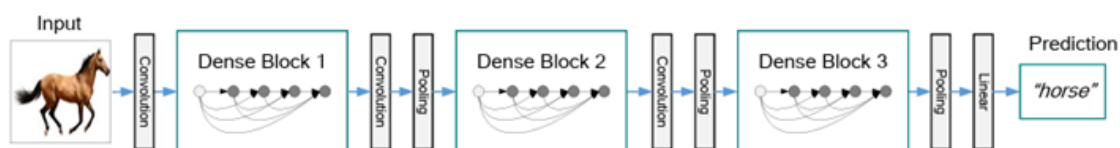
Zastosowanie tej techniki pozwoliło na uzyskanie modelu, który osiąga wyniki lepsze niż inne modele dostępne na rynku w trakcie pierwotnego badania, przy mniejszej ilości parametrów oraz wymagań dotyczących zasobów obliczeniowych.[34]

4.5. DenseNet

DenseNet to rodzina modeli konwolucyjnych sieci neuronowych (CNN), opracowana przez naukowców z Uniwersytetu Kalifornijskiego w Merced. Podobnie jak w modelach ResNet, zaimplementowano w niej formę połączeń skrótowych między warstwami, jednak w innej formie. Celem twórców było zaprojektowanie architektury, która sprostą występującemu problemowi zanikającego gradientu, który stanowił istotne wyzwanie w podczas trenowania głębokich sieci neuronowych. Twórcy tym samym dążyli do redukcji czasu klasyfikacji, aby umożliwić zastosowanie modelu do analizy wideo w czasie rzeczywistym.

W klasycznej sieci konwolucyjnej każda warstwa otrzymuje dane wyjściowe z poprzedzającej ją warstwy i przekształca ją we własną reprezentację cech. W przypadku bardzo głębokich sieci neuronowych, pojawia się problem zanikającego gradientu. Przez to, klasyczne sieci konwolucyjne nie mogą być zbyt głębokie, co może prowadzić do problemów z efektywnym uczeniem parametrów modelu.

Architektura DenseNet rozwiązuje ten problem w podobny sposób jak model ResNet. Wprowadza ona ideę gęstych połączeń. W DenseNet, każda warstwa znajdująca się w ramach jednego bloku nazwanego blokiem złożonym, otrzymuje wszystkie wyniki z poprzedzających ją warstw w postaci tensora.



Rysunek 4.5. Struktura modelu DenseNet. [35]

Założmy, że x_0 to dana wejściowa do bloku złożonego, $x_1, x_2, \dots, x_{l-1}$ to dane wyjściowe z kolejnych warstw bloku, a H_l to oczekiwana funkcją odwzorowania.

$$x_l = H_l([x_0, x_1, \dots, x_{l-1}])$$

gdzie, $[x_0, x_1, \dots, x_{l-1}]$ oznacza powiązanie (z ang. concatenation) wyników poprzedzających warstw. Dzięki temu model na każdej warstwie ma dostęp do całości danych, od danych wejściowych, po reprezentacje przetworzone przez kolejne warstwy. Pozwala to na dodawanie nowych cech, bez tracenia wcześniej wyuczonych informacji.[35]

5. Zbiór danych

Do przeprowadzenia badania wybrano zbiór danych PlantVillage-Dataset. Wybór tego zbioru wynikał z kilku czynników istotnych z punktu widzenia planowanych badań. Zdjęcia zawarte w zbiorze nie zawierają dodatkowych przekształceń i cechują się dobrą jakością, dzięki czemu możliwe było przeprowadzenie procesu przygotowania i modyfikacji danych zgodnie z założeniami jednego z eksperymentów. Istotnym argumentem była też struktura zbioru. Dane zostały odpowiednio uporządkowane i opisane, co ograniczało konieczność wykonania dodatkowych prac związanych z ich uporządkowaniem. Dodatkowym czynnikiem przejawiającym za wyborem tego zbioru było jego wykorzystanie w badaniach przedstawionych w przeglądzie literatury.

5.1. Charakterystyka zbioru danych

Zbiór danych zawiera 54 305 zdjęć liści roślin uprawnych wykonanych w środowisku laboratoryjnym. Posiada on 38 klas, które obejmują zdrowe okazy różnych gatunków roślin oraz chore okazy, wraz ze wskazaniem konkretnej choroby. Zdjęcia te zostały wykonane w warunkach laboratoryjnych, co może nie w pełni odzwierciedlać rzeczywiste warunki takie jak światło czy tło jakie występują w środowisku naturalnym. Niemniej jednak, pozwala on na przeprowadzanie kontrolowanej analizy i zrozumienie badanych zależności oraz na porównanie wyników z badaniami przedstawionymi w ramach przeglądu literatury



Rysunek 5.1. Przykładowe zdjęcia ze zbioru danych. [7]

5.2. Analiza zestawu

Klasa o najmniejszej liczba zdjęć	Potato___healthy
Klasa o największej liczba zdjęć	Orange___Haunglongbing_(Citrus_greening)
Największa liczba zdjęć dla klasy	5507
Najmniejsza liczba zdjęć dla klasy	152
Odchylenie standardowe	1254.8938177261505
Średnia liczba zdjęć na klasę	1429.078947368421

Tabela 5.1. Statystyki liczby zdjęć w zbiorze danych

Analiza statystyczna sugeruje niezrównoważony rozkład zdjęć w zbiorze danych. Jak widać z tabeli 5.1, istnieje duża różnica między klasą o największej liczbie zdjęć - Orange_Haunglongbing_(Citrus_greening), a klasą o najmniejszej liczbie zdjęć - Potato_healthy. Dodatkowo występuje duże odchylenie standardowe, bardzo bliskie średniej liczbie zdjęć na klasę. Wszystko to wskazuje na potrzebę zastosowania augmentacji danych, w celu zapobiegnięcia przeuczeniu.

6. Metodyka badań

Celem poniższego rozdziału jest przedstawienie specyfikacji sprzętowej, metod, narzędzi oraz przeprowadzonych badań porównawczych modeli głębokiego nauczania.

6.1. Środowisko badawcze

Wszystkie szkolenia modeli zostały wykonane na urządzeniu o podanych niżej specyfikacjach, w celu zachowania obiektywności eksperymentu.:

- Karta graficzna
 - Model: NVIDIA GeForce RTX 5080
- Procesor CPU:
 - Model: AMD Ryzen 9 9950X3D
- System operacyjny:
 - Nazwa systemu: Windows 11 Pro
 - Wersja systemu: 25H2
 - Typ systemu: 64-bitowy system operacyjny
- Pamięć RAM : 32 GB

Do testowania modeli wykorzystano komputer osobisty z konkretnymi specyfikacjami:

- Karta graficzna
 - Model: NVIDIA GeForce RTX 3050 Ti Laptop GPU
- Procesor CPU:
 - Model: AMD Ryzen 5 5600H with Radeon Graphics
- System operacyjny:
 - Nazwa systemu: Windows 11 Home
 - Wersja systemu: 21H2
 - Typ systemu: 64-bitowy system operacyjny
- Pamięć RAM : 16 GB

Od strony programowej implementację procesu treningowego wykonano w języku Python, z wykorzystaniem bibliotek przeznaczonych do przetwarzania obrazów oraz uczenia głębokich sieci neuronowych. Do implementacji trenowania modeli wykorzystano bibliotekę PyTorch wraz z modułem Torchvision, zapewniającym implementację analizowanych architektur oraz operacji przetwarzania i augmentacji obrazów. Biblioteki OpenCV oraz NumPy zostały wykorzystane do przetwarzania obrazów. Dodatkowo, wykorzystano standardowe moduły języka Python, między innymi do obsługi katalogów.

6.2. Lista modeli wykorzystana w badaniu

Dobór modeli do badania został przeprowadzony z uwzględnieniem ich skali oraz złożoności. Wybrano po dwa modele z każdej rodziny, różniące się rozmiarem, głębokością czy konfiguracją architektury. Takie podejście pozwoliło ograniczyć ryzyko wyciągnięcia wniosków dotyczących całej rodziny architektur na podstawie wyników uzyskiwanych przez pojedynczy model. Liczba oraz warianty wybranych modeli zostały jednocześnie ograniczone przez dostępne zasoby obliczeniowe, ponieważ zwiększenie liczby modeli wiązałoby się ze znacznym wydłużeniem czasu trenowania oraz zwiększeniem zapotrzebowania na pamięć i moc obliczeniową. W przypadku rodziny EfficientNet dodatkowym kryterium wyboru był rozmiar obrazu wejściowego poszczególnych wariantów. Wybrano modele EfficientNetB0 oraz EfficientNetB1, dla których rekomendowane rozmiary obrazów wejściowych to odpowiednio 224 x 224 oraz 240 x 240. Poniżej przedstawiono listę modeli wybranych do przeprowadzenia badania:

Rodzina modeli	Nazwa modelu	Typ architektury
ResNet	ResNet50	CNN
ResNet	ResNet101	CNN
ViT	ViT-B/16	ViT
ViT	ViT-B/32	ViT
MobileNet	MobileNetV2	CNN
MobileNet	MobileNetV3Small	CNN
EfficientNet	EfficientNetB0	CNN
EfficientNet	EfficientNetB1	CNN
DenseNet	DenseNet121	CNN
DenseNet	DenseNet169	CNN

Tabela 6.1. Lista modeli wykorzystanych w badaniu

6.3. Podział zbiorów danych

W ramach badania oryginalny zbiór danych został podzielony na trzy podzbiory: zbiór treningowy, zbiór walidacyjny oraz zbiór testowy. Następnie zbiór treningowy został wykorzystany do stworzenia kolejnych zbiorów. Zostały utworzone zbiory zawierające coraz mniejszą liczbę danych, oraz zbiory zawierające coraz więcej danych z rozmyciem ruchu (z ang. motion blur) oraz szumem gaussowskim (z ang. Gaussian noise).

Zbiór	Liczba zdjęć
Treningowy - 100 % danych	42 429
Treningowy - 50 % danych	21 214
Treningowy - 25 % danych	10 849
Treningowy - 25 % danych zanieczyszczonych	42 429
Treningowy - 50 % danych zanieczyszczonych	42 429
Treningowy - 75 % danych zanieczyszczonych	42 429
Testowy	5 459
Walidacyjny	5 417

Tabela 6.2. Lista zbiorów oraz liczba zdjęć

6.3.1. Dane z różnym procentem uszkodzonych danych

W ramach jednego z eksperymentów, zbiór treningowy został wykorzystany do przygotowania trzech nowych zbiorów treningowych zawierających coraz większy procent danych, które zostały poddane przekształceniom mającym na celu symulację słabszej jakości zdjęć. Zdjęcia zostały poddane poniższym przekształceniom:

- Rozmycie ruchu (z ang. motion blur) - efekt wizualny, który pojawia się kiedy robiąc zdjęcie obiektowi, ten wykonuje ruch. Objawia się to smugami i lekko rozmytym wyglądem,
- Szum gaussowski (z ang. gaussain noise) - to losowy szum podlegający rozkładowi normalnemu, czyli jego wartości skupiają się wokół średniej.

Poniżej przedstawiono przykładowe zdjęcia ze zbioru, na których dokonano przekształceń.



Rysunek 6.1. Przykładowe zdjęcia ze zbioru danych [7] na których dokonano przekształceń.

Parametr określający intensywność rozmycia ruchu był losowany dla każdego obrazu z zakresu 5-30 pikseli, a kierunek rozmycia losowo wybierany spośród dwóch opcji: poziomo, bądź pionowo. Do rozmytego obrazu dodawany następnie szum Gaussowski o losowo dobieranym odchyleniu standardowym z zakresu od 5 do 20.

6.4. Procedura treningu

Każdy z analizowanych modeli był trenowany niezależnie dla każdego przygotowanego wariantu zbioru danych. W celu zapewnienia porównywalności eksperymentów zastosowano jednolitą procedurę treningową, a różnicę pomiędzy poszczególnymi eksperymentami wynikały z charakterystyki zbioru treningowego.

W pierwszym etapie model był inicjalizowany za pomocą wag wstępnie wytrenowanych na zbiorze ImageNet-1K. Oryginalną warstwę klasyfikacyjną zastępowano warstwą dostosowaną do badanego zadania, które obejmowało 38 klas. Parametry całej sieci pozostawiono aktywne, co znaczy, że podczas treningu dokonywano dostrajania wszystkich wyszkolonych wag.

Przed rozpoczęciem każdego treningu ustalano generator liczb pseudolosowych na wartość 30. Obrazy ze zbioru treningowego poddawano augmentacji danych obejmujących zmianę wielkości, przycięcia, czy zmianę kontrastu. Następnie obraz był dostosowywany do wymagań danych wejściowego dla danego modelu. Zbiory testowe, oraz walidacyjne nie były poddawane losowym operacjom augmentacyjnym.

Trening był przeprowadzony z wykorzystaniem optymalizatora Adam, funkcją straty Cross-Entropy, oraz współczynnikiem uczenia $1e-4$. Rozmiar próbki (z ang. batch) został ustalony na 128. Podczas treningu wykorzystano technikę precyzji mieszanej, która pozwoliła na efektywniejsze wykorzystanie zasobów sprzętowych oraz krótsze szkolenia modeli. Liczbę epok ustalono na 50.

Po zakończeniu każdej epoki, model był poddawany ocenie z wykorzystaniem zbioru walidacyjnego. Obliczano funkcję straty oraz dokładność klasyfikacji. Wartość straty na zbiorze walidacyjnym była następnie wykorzystywana do wyboru najlepszego stanu modelu. Jeśli wartość była niższa, niż w poprzednich epokach, aktualny stan był zapisywany jako najlepszy.

Aby ograniczyć długość treningu, i zapobiec przeuczeniu, wykorzystano technikę Wczesnego zatrzymywania. Jeśli przez pięć kolejnych epok model nie poprawił swoich wyników, trening był zatrzymywany.

Po zakończeniu treningu, najlepszy model, ustalony poprzez ocenę go na zbiorze walidacyjnym, był poddawany ocenie na zbiorze testowym.

6.5. Konfiguracja treningu

Niniejszy podrozdział przedstawia ogólną konfigurację modeli wykorzystaną w przeprowadzonych eksperymentach. Opisano w nim przeprowadzoną augmentację danych, wybrane parametry oraz zastosowane techniki w ramach przeprowadzonych eksperymentów. Zastosowanie jednolitej konfiguracji pozwala na bardziej rzetelne i obiektywniejsze porównanie badanych modeli.

6.5.1. Augmentacja danych

Z powodu niezbalansowanego zbioru danych zastosowano augmentację danych, której celem jest zwiększenie różnorodności danych treningowych. Zdjęcia dostarczane do modeli w ramach zbioru treningowego są poddawane przekształceniom, takim jak:

- Losowe odwracanie - zdjęcia zostały lekko obrócone
- Odwracanie poziome - zdjęcia zostały obrócone poziomo tzw. odbicie lustrzane
- Losowy kontrast - zdjęciom zostało losowo podbity kontrast
- Losowe przybliżenie - zdjęcia zostały losowo przybliżone, co prowadzi do przycięcia niektórych elementów obiektu

Warstwa ta działa tylko podczas uczenia i nie wpływa na obrazy przekazane w celu sprawdzenia skuteczności modelu, czy w celu klasyfikacji.



Rysunek 6.2. Przykładowe zdjęcie ze zbioru danych [7] po poddaniu go augmentacji danych.

Na Rys. 6.2 przedstawiono przykładowe rezultaty zastosowania augmentacji danych na jednym ze zdjęć. W trakcie treningu transformacje były dobierane losowo, dlatego rzeczywisty obraz wejściowy mógł być poddany innej kombinacji operacji.

Operacja	Parametr	Wartość
Resize	Rozmiar wyjściowy	$IMG_SIZE \times IMG_SIZE$
RandomHorizontalFlip	Prawdopodobieństwo	0,5
RandomRotation	Zakres rotacji	$\pm 20^\circ$
ColorJitter	Zmiana kontrastu	0,2
RandomResizedCrop	Zakres skali	0,8–1,0

Tabela 6.3. Operacje i parametry stosowane podczas przygotowania danych treningowych

W powyższej tabeli przedstawiono parametry poszczególnych operacji.

Dobór operacji augmentacyjnych oraz ich parametrów miał na celu zwiększenie różnorodności danych treningowych przy jednoczesnym zachowaniu istotnych cech wizualnych obrazów. Zastosowano transformacje, które nie prowadzi do istotnego zniekształcenia przedstawionych obiektów, a jednocześnie wprowadzają zauważalne różnice względem obrazów oryginalnych.

6.5.2. Parametry

Parametry w procesie treningowym zostały ujednolicone dla każdego modelu, aby ograniczyć wpływ zmiennych zakłócających i zapewnić porównywalność i wiarygodność uzyskanych wyników. Poniższe parametry zostały wybrane również ze względu na ograniczenia sprzętowe:

- Optymalizator: Adam
- Liczba epok: 50
- Rozmiar partii: 128
- Rozmiar zdjęć: 224x224 i 240x240
- Współczynnik uczenia: $1e-4$
- Ziarno: 30
- Wagi: IMAGENET1K_V1

Ustalono maksymalną liczbę epok na 50, jednocześnie stosując mechanizm wczesnego zatrzymywania (z ang. Early Stopping) monitorujący wartość funkcji straty na zbiorze walidacyjnym. W praktyce trening modeli kończył się przed osiągnięciem maksymalnego limitu, przez co liczba 50 epok stanowiła górne ograniczenie czasu uczenia, a nie wymóg wykonania pełnych 50 epok.

W celu ograniczenia wpływu czynników losowych podczas eksperymentów ustalono wartości ziarna generatorów liczb losowych na 30. Wartość ta była stosowana jednolicie we wszystkich eksperymentach.

Jako optymalizator został zastosowany Adam. Jest to popularne rozwiązanie, szeroko stosowane w trenowaniu głębokich sieci neuronowych. W celu zapewnienia jednolitych warunków porównania wszystkie analizowane architektury trenowano z wykorzystaniem tego samego optymalizatora.

Współczynnik uczenia ustalono na poziomie 1-4e. Wartość tę przyjęto jako stały hiperparametr treningu, zgodnie z założeniem przeprowadzenia porównania architektur w jednolitych warunkach.

Rozmiar partii (z ang. batchy) został ustalony na 128 próbek, uwzględniając możliwości sprzętowe wykorzystywanego środowiska obliczeniowego. Wartość ta pozwalała na efektywne wykorzystanie dostępnych zasobów pamięci GPU i skrócenie czasu realizacji eksperymentów, co miało znaczenie ze względu na liczbę przeprowadzonych treningów.

W celu zachowania jednolitych warunków eksperymentalnych dla większości analizowanych modeli zastosowano rozdzielczość wejściową 224x224 piksele. Wyjątek stanowił model EfficientNetB1, dla którego zastosowano rozdzielczość 240x240 pikseli, zgodnie z konfiguracją tego wariantu architektury. Zróżnicowanie rozdzielczości wynikało z charakterystyki analizowanych modeli i nie było elementem badanego czynnika eksperymentalnego.

Wszystkie analizowane modele zostały zainicjalizowane wagami wstępnie wytrenowanymi na zbiorze ImageNet. Wybór wag ImageNet wynikał z dostępności jednolitego wariantu wag dla wszystkich analizowanych architektur.

6.5.3. Wczesne zatrzymywanie

Wczesne zatrzymywanie (z ang. EarlyStopping) to technika zapobiegająca przeuczeniu modeli w trakcie uczenia. Polega ona na monitorowaniu wybranej metryki. W momencie zaobserwowania braku poprawy w wybranym okresie trening jest zatrzymywane, a wagi modelu zapisywane. Poza zapobieganiem przeuczeń modeli, technika ta pozwoli też na zmniejszenie czasu potrzebnego na wytrenowanie jednego modelu.

W przeprowadzonym treningu, Wczesne zatrzymywanie brało pod uwagę pięć kolejnych epok - zatem jeśli przez pięć następujących po sobie epok model nie poprawi się, trening jest zatrzymywany.

W ramach treningu monitorowano błąd modelu na danych walidacyjnych. Zbiór walidacyjny, niezależny od innych modeli, był używany pod koniec każdej epoki, w celu oceny stanu modelu. Jeśli aktualny stan parametrów okazywał się być lepszy, niż poprzedni najlepszy, plik z wagami był zapisywany, licznik epok resetowany do zera, a błąd na zbiorze walidacyjnym zapisywany. Jeśli natomiast błąd był wyższy, licznik epok szedł w górę, a trening trwał dalej. Trening zatrzymywał się w momencie osiągnięcia przez licznika wskazanej liczby epok. Finalny zapis wag był dokonywany zatem pięć epok przed końcem treningu.

Zastosowanie tej techniki pozwoliło ograniczyć czas obliczeń, a jednocześnie ograni-

czał ryzyko nadmiernego dopasowania modelu do danych treningowych, które mogłyby wystąpić przy dłuższym treningu.

6.5.4. Precyzja mieszana

Precyzja mieszana (z ang. mixed precision) to użycie 16 bitowych oraz 32-bitowych typów zmiennoprzecinkowych w modelu podczas uczenia, w celu przyspieszenia jego działania oraz ograniczenia zużycia pamięci. Pozwala to na skrócenie czasu kroku, przy zachowaniu jakości treningu. Standardowo, większość modeli trenowanych jest przy użyciu 32-bitowych liczb zmiennoprzecinkowych, co pozwala na osiągnięcie wysokiej skuteczności, co idzie jednak z kosztem dużego użycia pamięci oraz długiego czasu treningu. Precyzja mieszana natomiast używa jednocześnie 16 bitowych oraz 32 bitowych liczb zmiennoprzecinkowych. 32 bitowe liczby są używane do krytycznych operacji, które wymagają dużej precyzji, a 16-bitowe do operacji, gdzie mniejsza precyzja jest akceptowalna. Zastosowanie tej techniki pozwala na skrócenie czasu trenowania oraz zmniejszenie zużycia pamięci, bez wpływu na jakość przeprowadzonego treningu.[36]

6.6. Wykonane eksperymenty

W ramach przeprowadzonych badań wykonano dwa eksperymenty, których celem było określenie wpływu charakterystyki używanego zbioru treningowego na skuteczność analizowanych modeli. W pierwszym eksperymencie zbadano wpływ liczby danych treningowych, a w drugim eksperymencie wpływ jakości danych na otrzymane wyniki.

Oba eksperymenty przeprowadzono z wykorzystaniem tej samej strategii treningowej oraz konfiguracji hiperparametrów, z wyjątkiem modelu EfficientNetB1, gdzie z powodu specyfiki tego modelu, należy zastosować inną rozdzielczość wejściową.

6.6.1. Trenowanie modeli na zbiorach z różną liczbą danych

Celem tego eksperymentu było zbadanie wpływu liczby danych w zbiorze treningowym na skuteczność klasyfikacji analizowanych architektur. Założeniem eksperymentu było sprawdzenie, czy coraz mniejsza liczba dostępnych danych w trakcie treningu jednakowo wpłynie na badane modele, czy też niektóre z nich wykazują lepszą odporność na to ograniczenie.

W ramach eksperymentu każdy z 10 modeli został wytrenowany na trzech wariantach zbioru treningowego. Pierwszy wariant wykorzystywał pełen zbiór danych, drugi tylko 50% oryginalnego zbioru, a trzeci 25%. Każdy trening został wykonany niezależnie, to znaczy, że poszczególne treningi nie wpływały na wyniki innych. Łącznie zostało wykonanych 30 treningów.

6.6.2. Trenowanie modeli na zbiorach z różnym procentem uszkodzonych danych

Celem tego eksperymentu było zbadanie wpływu jakości danych w zbiorze treningowych na skuteczność klasyfikacji analizowanych architektur. Założeniem eksperymentu

było sprawdzenie, czy coraz większy udział obrazów poddanych kontrolowanemu zniekształceniu wpływa na jakość klasyfikacji i w jaki sposób wśród badanych modeli.

Każdy z 10 modeli został wytrenowany na trzech zbiorach. Każdy ze zbiorów posiadał wszystkie dane z oryginalnego zbioru treningowego. Różnią się one natomiast liczbą zniekształconych danych. W pierwszym zbiorze udział obrazów poddanych zniekształceniom wynosi 75 %, w drugim 50%, w trzecim 25%. Każdy trening został wykonany niezależnie, to znaczy, że poszczególne treningi nie wpływały na wyniki innych. Łącznie zostało wykonanych 30 treningów.

6.7. Użyte miary

Pomimo przedstawienia w rozdziale wielu miar wykorzystywanych do oceny jakości modeli klasyfikacji, w ramach analizy wyników, wykorzystano jako główną metrykę oceny miarę Macro F1-Score. Wybór ten został podyktowany naturą badania. W ramach trenowania modeli został użyty zbiór wieloklasowy, który, jak wynika z analizy przedstawionej w rozdziale , nie ma równomiernego rozkładu zdjęć między klasami. W związku z tym, Macro F1 jest idealną miarą badawczą, która bierze pod uwagę fakt nierówności klas, a jednocześnie łączy w sobie miary Precision oraz Recall.

7. Wyniki

7.1. Charakterystyka badanych modeli

Poniższa tabela prezentuje charakterystykę modeli wykorzystanych w eksperymentach. Dane z tabeli pochodzą z pierwszego eksperymentu, konkretnie z modeli wyszkolonych na całym zbiorze danych.

Nazwa modelu	Średni czas na przetworzenie jednej partii 16 zdjęć	Liczba parametrów	Rozmiar modelu po szkoleniu
ResNet50	0.0124	23585894	90,2 MB
ResNet101	0.0224 s	42578022	163 MB
ViT-B/16	0.0097 s	85827878	327 MB
ViT-B/32	10.0108 s	87484454	333 MB
MobileNetV2	0.0109 s	2272550	8,90 MB
MobileNetV3Small	0.0106 s	1556806	6,06 MB
EfficientNetB0	0.0156 s	4056226	15,7 MB
EfficientNetB1	0.0226 s	6561862	25,4 MB
DenseNet121	0.0321 s	6992806	27,2 MB
DenseNet169	0.0451 s	12547750	48,8 MB

Tabela 7.1. Statystyki modeli dla tego samego zbioru danych

7.2. Eksperyment 1 - Skalowanie efektywności względem ilości danych treningowych

Poniższa tabela prezentuje wyniki uzyskane podczas szkolenia modeli na zbiorach danych o różnej wielkości oraz jednakowej konfiguracji.

Nazwa modelu	Macro F1-Score		
	100% danych	50 % danych	25 % danych
ResNet50	0.9941	0.9927	0.9751
ResNet101	0.9961	0.9869	0.9860
ViT-B/16	0.9928	0.9867	0.9803
ViT-B/32	0.9846	0.9859	0.9705
MobileNetV2	0.9925	0.9902	0.9866
MobileNetV3Small	0.9921	0.9801	0.9775
EfficientNetB0	0.9937	0.9956	0.9855
EfficientNetB1	0.9928	0.9893	0.9869
DenseNet121	0.4530	0.6234	0.7676
DenseNet169	0.7042	0.7032	0.8275

Tabela 7.2. Wyniki klasyfikacji dla zbiorów treningowych o różnej wielkości

7.3. Eksperyment 2 - Skalowanie efektywności względem ilości zaszumionych danych

Poniższa tabela prezentuje wyniki uzyskane podczas szkolenia modeli na zbiorach o jednakowej wielkości, z różną ilością danych treningowych poddanych przekształceniu oraz jednakowej konfiguracji.

Nazwa modelu	Macro F1-Score			
	czysty	75%	50%	25%
ResNet50	0.9941	0.9941	0.9943	0.9941
ResNet101	0.9961	0.9960	0.9931	0.9927
ViT-B/16	0.9928	0.9890	0.9961	0.9803
ViT-B/32	0.9846	0.9840	0.9905	0.9837
MobileNetV2	0.9925	0.9950	0.9931	0.9967
MobileNetV3Small	0.9921	0.9987	0.9910	0.9905
EfficientNetB0	0.9937	0.9953	0.9956	0.9962
EfficientNetB1	0.9946	0.9947	0.9945	0.9942
DenseNet121	0.4530	0.4836	0.5013	0.5265
DenseNet169	0.7042	0.5799	0.5717	0.5581

Tabela 7.3. Wyniki klasyfikacji dla różnego procenta zaszumienia zbioru treningowego

8. Analiza wyników

8.1. Analiza wpływu danych

Analiza wyników wskazuje, że zmniejszanie liczebności danych w zbiorze treningowy, minimalnie wpłynęło na pogorszenie skuteczności klasyfikacji. Dla większości modeli wartości miary Macro F1 były coraz mniejsze, im mniej danych otrzymywał model w trakcie nauki. Różnice między wynikami dla zbiorów treningowych z różną ilością danych były jednak małe - różnica wynosi około 1 punktu procentowego. Zatem, mimo zaobserwowania wpływu liczby danych treningowych na jakość klasyfikacji, większość badanych modeli jest w stanie osiągnąć wysokie miary, mimo coraz mniejszego zbioru danych. Tak wysokie wyniki mogą być spowodowane, poza samą specyfiką zadania, w której badane modele osiągają wysokie miary, jakością zbioru użytego w badaniu - zdjęcia są wysokiej jakości, wykonane na tym samym tle, z dobrze widocznym obiektem do identyfikacji. Dobrej jakości zbiór danych mógł pozwolić modelom na skupienie się na nauce cech, które różnią klasy, a nie braniu pod uwagę, na przykład, oświetlenia, tła czy zakłóceń.

Najlepiej ze zmniejszaniem liczby danych treningowych poradziły sobie modele MobileNetV2, EfficientNetB0 oraz EfficientNetB1. Mają najmniejszą różnicę miary Macro F1 między coraz mniejszymi zbiorami treningowymi - są one zatem najbardziej stabilne z całej grupy badawczej.

Najlepsze wyniki w zestawieniu osiągnęły modele ResNet. Udało im się to osiągnąć jedynie dla szkoleń wykonanych na całym zbiorze. Miały one większą różnicę niż modele z rodziny EfficientNet, mimo posiadania większej ilości parametrów. Ich przewaga nad modelami z rodziny EfficientNet oraz MobileNet była minimalna, zatem, mimo tego, że model ResNet101 osiągnął najlepszy wynik, nie stanowi on dobrego kompromisu między skutecznością, a wymaganiami pamięciowymi, co odróżnia go od najbardziej stabilnych modeli.

Modele z rodziny MobileNet mają najmniej parametrów w zestawieniu osiągając podobne, a jak w przypadku modelu MobileNetV2, najlepsze wyniki w porównaniu z badanymi modelami. Mimo, widocznego spadku jakości klasyfikacji dla modelu w wersji MobileNetV3Small przy zmniejszeniu danych, rodzina tych modeli stanowi kompromis między skutecznością klasyfikacji, a wymaganiami pamięciowymi.

Najgorzej z całym zadaniem poradziły sobie modele z rodziny DenseNet. Oba modele, Densenet121 i DenseNet169 osiągnęły zdecydowanie niższe wyniki na każdym z trzech badanych zbiorów od reszty badanych modeli. To jedyne modele, które osiągały lepsze wyniki z mniejszą ilością danych. Nie wynika to jednak prawdopodobnie z błędu podczas przeprowadzania eksperymentu - oba modele wykazują takie samo zachowanie, a reszta modeli była szkolona w ten sam sposób, w tym samym środowisku oraz na tych samych

danych. Prawdopodobnie, DenseNet nie radzi sobie z dużą ilością danych, które prowadzą do przeuczenia - wyjaśniałoby to czemu na małym zbiorze oba modele radzą sobie najlepiej. Modele te jako jedyne nie poradziły sobie z zadaniem, gdzie każdy z wyników otrzymanych dla różnych zbiorów odbiega znacząco od wyników reszty badanych modeli.

Modele z rodziny ViT osiągnęły gorsze wyniki od modeli CNN, pomijając modele DenseNet. Mimo stabilności oraz małego spadku jakości pomiędzy badanymi zbiorami, modele te osiągają gorsze wyniki niż reszta modeli. Można z tego wywnioskować, że modele te radzą sobie gorzej z tym zdaniem, co może być spowodowane, przez zbiór danych, w którym zdjęcia dostarczone do szkolenia oraz walidacji i testowania modeli są do siebie bardzo podobne, gdyż różnią się małymi szczegółami. Można zauważyć, że model w wersji ViT-B/16 osiąga lepsze wyniki niż model w wersji ViT-B/32. Może to wynikać z faktu, że obraz jest dzielony na więcej fragmentów, co pozwala modelowi na lepszą naukę cech. Modele te, są też największe w zestawieniu - mają najwięcej parametrów, jak i największy rozmiar. Mimo tego, ich klasyfikacja jest mniej dokładna od modeli CNN, które są zdecydowanie mniejsze, co udowadnia, że więcej parametrów nie przekłada się na lepszy wynik.

8.2. Analiza wpływu zanieczyszczeń w danych

Analiza wyników wskazuje, że zwiększanie udziału zaszumionych obrazów w zbiorze treningowym, nie wpłynęła do jednoznacznego pogorszenia skuteczności klasyfikacji. Dla większości modeli wartości miary Macro F1 pozostawały na podobnym poziomie, niezależnie od procenta zaszumionych danych w zbiorze szkoleniowym. Wyniki wskazują na to, że badane modele charakteryzują się wysoką odpornością na umiarkowane pogarszanie jakości danych szkoleniowych i są w stanie dokonać poprawnej klasyfikacji, mimo obecności zakłóceń, na prawie takim samym poziomie, co używając modeli szkolonych na czystym zbiorze.

Najbardziej odporne na zaszumienie były modele ResNet50 oraz EfficientNetB1, gdzie różnica między wynikami, dla różnego procentu zaszumionych danych w zbiorze treningowym wynosiła zaledwie kilka tysięcznych punktu Macro F1. Zaszumione dane praktycznie nie wpłynęły na jakość klasyfikacji.

Modele CNN, pomijając rodzinę DenseNet, utrzymują wyniki na podobnym poziomie, bez względu na ilość zaszumionych danych. Mimo zaobserwowania, widocznych różnic w wynikach pomiędzy zbiorami, różnice te są bardzo małe. Z tego powodu należy zwrócić uwagę na charakterystykę badanych modeli. Modele z rodziny EfficientNet oraz MobileNet stanowią idealny kompromis, między odpornością na zaszumienia, a wielkością tych modeli. Obie rodziny mają mniej parametrów niż reszta modeli w zestawieniu, a mimo tego, osiągają podobne wyniki.

Warto także zwrócić uwagę na fakt, że dla większości modeli, z wyjątkiem DenseNet169 oraz MobileNetV3Small, dodanie zaszumionych danych pomogło modelom uzyskać więk-

szą miarę Macro F1. Dla niektórych, było to 50%, dla innych 75%. Może to sugerować, że dodanie danych z artefaktami osiągnęło podobny efekt do augmentacji, ograniczając tym samym ryzyko nadmiernego dopasowania modelu do zbioru treningowego oraz poprawiając jego zdolność do generalizacji. Można to zjawisko interpretować jako zwiększanie różnorodności danych treningowych.

Modele z rodziny DenseNet poradziły sobie najgorzej z problemem zaszumionych danych. Uzyskały one najniższe miary Macro F1. Najgorsze wyniki ze wszystkich badanych architektur, uzyskał model DenseNet121. Warto jednak zwrócić uwagę na fakt, że, tak jak reszta modeli, można zaobserwować, że zaszumione dane spełniają rolę augmentacji. Model ten uzyskuje najlepszy wynik dla zbioru szkoleniowego z 25% zaszumionych danych, gdzie reszta zbiorów treningowych także osiąga lepsze wyniki od czystego zbioru. Różnice między wynikami dla tego modelu są także większe, niż dla reszty modeli jednak, nadal można stwierdzić, że model ten jest stabilny. Mimo to, model ten osiąga zdecydowanie gorsze wyniki od reszty modeli, jako jedyny osiągając wynik poniżej 0.5 Macro F1 - co sugeruje, że model ten nie radzi sobie z tym problemem. To samo można stwierdzić na temat DenseNet169, tutaj jednak można zaobserwować znacznie większą różnicę, między wynikami uzyskanymi dla czystego zbioru, a zbiorami zaszumionymi. Model ten ma największą różnicę między wynikami, gdzie osiąga najlepsze wyniki dla czystego zbioru - tak samo jak drugi model z rodziny nie radzi sobie z tym problem, jednak wpływ zaszumionych danych jest największy dla tego modelu i jest jednoznacznie negatywny.

Modele z rodziny ViT-B są stabilne. Występuje mała różnica między wynikami, gdzie pomijając modele z rodziny DenseNet, różnica ta jest zdecydowanie większa, niż dla modeli CNN. Można z tego wywnioskować, że modele bazowane na transformerach mają mniejszą odporność na zakłócenia w danych. Nie można jednak powiedzieć, że otrzymane wyniki sugerują, że modele te nie radzą sobie z tym zadaniem - miara Macro F1 nadal jest wysoka, powyżej 0.95. Modele te, mimo wysokich wyników, osiągają jednak mniejsze miary niż modele CNN. Dodatkowo, mają one więcej parametrów, niż reszta modeli. W związku z tym, można stwierdzić, że modele z rodziny ViT radzą sobie gorzej, niż modele typu CNN, mimo większej ilości parametrów oraz większego rozmiaru. Oba modele z tej rodziny mają podobne wyniki, jednak model ViT-B/16 ma odrobinę lepsze wyniki od ViT-B/32. Może to wynikać z faktu, że wielkość fragmentów obrazów jaka była używana przy pierwszym modelu jest mniejsza, a co za tym idzie, model dzieli obraz na więcej fragmentów co mogło mu pomóc w lepszym poszukiwaniu cech.

8.3. Finalna ocena modeli

Analiza wyników obu eksperymentów wskazuje, że badane modele jednocześnie radziły sobie świetnie z coraz mniejszą liczbą danych oraz z coraz większą liczbą zniekształconych danych. W obu przypadkach wartości Macro F1 pozostawały na bardzo wysokim poziomie.

Najbardziej stabilnymi były modele z rodzin EfficientNet oraz MobileNet. Miały najmniejsze różnice między zbiorami w pierwszym eksperymencie, jak i drugim. Dodatkowo, są to modele z mniejszą liczbą parametrów, w porównaniu do innych badanych w ramach eksperymentów. Warto jednak podkreślić, że modele te nie osiągały najlepszych miar, jednak były one stabilne, przez co można stwierdzić, że są one kompromisem między skutecznością oraz odpornością na zakłócenia, a wymaganiami pamięciowymi.

Najlepsze wyniki osiągały modele z rodziny ResNet. Miara Posiadają one zdecydowanie więcej parametrów, niż np. modele z rodziny MobileNet, co nie przekłada się jednak na znaczną różnicę w jakości klasyfikacji przedstawionej przez tamte modele. Mimo to, że mają one najlepsze wyniki, modele te ważą zdecydowanie więcej niż modele, które osiągają podobne wyniki.

Najgorzej z zadaniami poradziły sobie modele z rodziny DenseNet. DenseNet osiągnął najgorsze wyniki w każdym eksperymencie, znacząco odbiegając od reszty modeli. Modele te są stabilne, jednak osiągają gorsze wyniki, niż na modelach uczących się na jednym ze zbiorów bez zaszumionych danych treningowych. Za tą rodziną nie przemawia też argument efektywności, gdyż są one większe pod względem rozmiaru od modeli z rodziny EfficientNet i MobileNet, które osiągają zdecydowanie lepsze wyniki.

ViT-B były jedynymi modelami opierających się na transformerach. W obu eksperymentach oba modele z tej rodziny osiągnęły wysokie wyniki, jednak nie poradziły sobie lepiej niż modele CNN. Osiągnęły one niższe wyniki w obu eksperymentach, będąc jedynie lepsze od modeli DenseNet. Były one stabilne, a różnica między wynikami dla zbiorów nie była duża, jednak nadal była większa, niż przy innych modelach. Modele z tej rodziny były także największe w zestawieniu. Z tych powodów nie można uznać, że modele z tej rodziny są lepsze od modeli CNN, choć radzą sobie z tym zadaniem, co dowodzi, że modele oparte na transformerach mogą znaleźć zastosowanie w klasyfikacji.

9. Aplikacja

Aplikacja stanowi praktyczne wykorzystanie rozwiązań badanych w ramach części badawczej. Jest ona demem technologicznym, którego celem jest demonstracja praktycznego zastosowania badanych modeli oraz przedstawienia możliwości wykorzystania uczenia maszynowego w aplikacji mobilnej.

9.1. Technologie

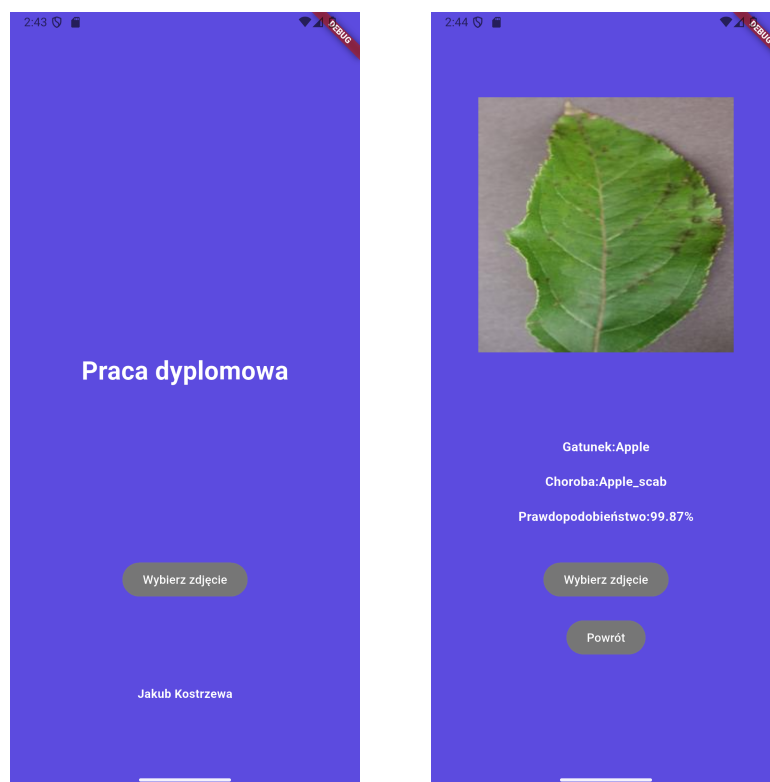
Aplikacja została zaimplementowana w języku Dart z wykorzystaniem środowiska Flutter. Do testowania aplikacji wykorzystano emulator dostępny w środowisku Android Studio symulujący telefon mobilny z systemem Android.. Do implementacji modelu wykorzystano bibliotekę TensorFlow. Zarówno TensorFlow, jak i Flutter są produktami od firmy Google, dzięki czemu są ze sobą kompatybilne i umożliwiają użycie modelu bezpośrednio na urządzeniu mobilnym, dzięki czemu aplikacja może wykonywać klasyfikację bez dostępu do sieci internetowej. Do konwersji modelu zapisanego w formacie *.pth* do formatu *.tf* wykorzystywanego przez TensorFlow został wykorzystany ekosystem wymiany modeli Onnx.

9.2. Implementacja modelu

W aplikacji wykorzystano model MobileNetV3Small. Model ten został zaprojektowany z myślą o zastosowaniach na urządzeniach mobilnych. Dodatkowo, na podstawie przeprowadzonych badań, udało się stwierdzić, że modele z tej rodziny stanowią dobry kompromis między skutecznością a wymaganiami sprzętowymi.

Model został przekonwertowany z wcześniej zapisanych wag w formacie *.pth* używanym przez bibliotekę PyTorch na format *.tf* używany przez TensorFlow. Do tego został użyty ekosystem Onnx. Dzięki temu model może być umieszczony bezpośrednio w plikach aplikacji, a sama klasyfikacja wykonywana lokalnie, bez potrzeby dostępu do serwera.

9.3. Widoki i działanie



Rysunek 9.1. Zdjęcie ekranu startowego oraz ekranu z wynikami

W ramach pierwszego widoku aplikacja pozwala użytkownikowi na kliknięcie przycisku *Wybierz zdjęcie*. Spowoduje to otwarcie galerii, gdzie użytkownik może wybrać zdjęcie przeznaczone do klasyfikacji.

W ramach widoku drugiego aplikacja wyświetla wyniki klasyfikacji uzyskane przez model oraz wybrane wcześniej zdjęcie. Użytkownik otrzymuje informację o gatunku rośliny, rozpoznanej chorobie oraz wartości prawdopodobieństwa poprawnie wykonanej klasyfikacji.

Na tym ekranie znajdują się też dwa przyciski - *Wybierz zdjęcie* oraz *Powrót*. Kliknięcie pierwszego przycisku spowoduje ponowny wybór zdjęcia oraz ponowne wyświetlenie, już nowych, wyników. Drugi przycisk pozwala na powrót do ekranu startowego aplikacji.

10. Podsumowanie

Celem pracy było zbadanie, jak wybrane architektury głębokiego uczenia różnią się pod względem skuteczności oraz odporności na redukcję liczby danych treningowych i degradację ich jakości w zadaniu klasyfikacji chorób roślin. Dodatkowym celem było stworzenie prototypu, w postaci aplikacji mobilnej, która jest przykładem wykorzystania badanego zagadnienia.

W ramach pracy przeprowadzono przegląd literatury dotyczącej klasyfikacji chorób roślin z wykorzystaniem metod uczenia maszynowego (roz. 6 i roz. 3). Na jego podstawie dokonano wyboru zbiorów danych oraz architektur poddanych dalszej analizie, obejmujących modele ViT, EfficientNet, MobileNet, ResNet, oraz DenseNet. Następnie zaprojektowano i przygotowano eksperymenty pozwalające na ocenę wpływu jakości oraz liczby danych na skuteczność badanych modeli. Przeprowadzono treningi poszczególnych architektur dla różnych wariantów danych, a uzyskane wyniki zostały poddane analizie porównawczej. Pozwoliło to na określenie różnic w skuteczności modeli oraz ich odporności na odgraniczenia związane z liczbą oraz jakością danych treningowych. Dodatkowo, przygotowano aplikacje mobilną wykorzystującą jeden z badanych modeli,

Na podstawie przeprowadzonych badań można stwierdzić, że większość analizowanych architektur biorących udział w badaniu jest skuteczna w ramach zadania klasyfikacji chorób roślin. Jedynym wyjątkiem jest rodzina modeli DenseNet, gdzie oba warianty osiągnęły znacznie gorsze wyniki od pozostałych badanych modeli w obu przeprowadzonych eksperymentach.

Na podstawie pierwszego eksperymentu ustalono, największą odporność na zmniejszanie liczby danych treningowe mają modele MobileNetV2, EfficientNetB0 oraz EfficientNetB1. Miały one najmniejszą różnicę miary Macro F1. Najlepszy wynik dla całego zbioru uzyskały modele ResNet, jednak miały one większe spadki jakości, niż wcześniej wspomniane modele. Najlepszym kompromisem między jakością klasyfikacji, odpornością na spadek liczby danych oraz wymaganiami pamięciowymi stanowiły modele z rodziny MobileNet, które osiągały zbliżone wyniki do modeli z większą ilością parametrów, a przy tym nie odbiegały jakością od innych analizowanych modeli.

Na podstawie drugiego eksperymentu ustalono, że największą odporność na zniekształcone dane treningowe mają modele ResNet50 oraz EfficientNetB1. Różnica między Macro F1 dla użytych zbiorów treningowych była dla nich najmniejsza i wynosiła zaledwie kilka tysięcznych. Najlepszym kompromisem, między jakością klasyfikacji, odpornością na spadek jakości danych oraz rozmiarem były modele z rodzin EfficientNet i MobileNet - ich wyniki były zbliżone do innych z zestawienia, a jednocześnie miały one zdecydowanie mniej parametrów.

W ramach finalnej analizy obu eksperymentów ustalono, że najlepszym kompromisem między wynikami obu eksperymentów, a wydajnością pamięciową są rodziny EfficientNet i MobileNet. Najlepsze wyniki osiągały modele z rodziny ResNet, a najgorzej z oboma

zadaniami radziły sobie modele z rodziny DenseNet. Nie zaobserwowano przewagi modeli opartych na transformerach nad modelami CNN - modele z rodziny ViT osiągały zbliżone, choć gorsze wyniki od modeli CNN. W ramach badania nie zaobserwowano także zależności między rozmiarem modelu a wynikami. Największe modele, z rodziny ViT, nie uzyskały przewagi nad modelami mniejszymi, a uzyskały wyniki gorsze.

10.1. Dalsze kierunki badań

Otrzymane rezultaty badań stanowią podstawę do dalszych badań nad doborem i optymalizacją architektur przedstawionych w badaniu. Poprzez zastosowanie identycznych parametrów podczas trenowania modeli uzyskano wyniki, które można ze sobą obiektywnie porównać, wybrane hiperparametry mogą nie być optymalne dla danych modeli. Interesującym kierunkiem byłoby sprawdzenie, czy odpowiednie dobranie hiperparametrów i zastosowanie dodatkowych technik augmentacji danych pozwoliłoby uzyskać przewagę modelowi ViT nad CNN. W przyszłych badaniach warto także zastosować inny zbiór danych - dane wykorzystane w tym badaniu są zebrane w jednym, kontrolowanym środowisku laboratoryjnym. Sprawdzenie, czy modele radzą sobie w detekcji chorób w środowisku naturalnym, gdzie zdjęcia mogą być wykonane pod różnym kątem, z różnym tłem, czy obiektami zasłaniającymi część liści pozwoliłoby ocenić, czy dane modele są w stanie znaleźć zastosowanie w warunkach rzeczywistych, bezpośrednio w gospodarstwie rolnym, a nie w kontrolowanym środowisku laboratoryjnym. Dalsze badania mogłyby także rozszerzyć zakres badanych modeli - badanie to obejmowało tylko 10 modeli, a analiza większej liczby architektur, w tym tych opartych o transformery, pozwoliłaby na szerszą ocenę oraz sprawdzenie, czy przewaga modeli CNN nad ViT utrzymuje się również w przypadku innych rozwiązań.

Bibliografia

- [1] Food and Agriculture Organization of the United Nations, *The Hidden Health Crisis: How Plant Diseases Threaten Global Food Security*, <https://www.fao.org/one-health/highlights/how-plant-diseases-threaten-global-food-security/en>, Dostęp zdalny (16.06.2026), 2025.
- [2] Polski Czerwony Krzyż, *Światowy Dzień Walki z Głodem*, <https://pck.pl/polski-czerwony-krzyz/aktualnosci/2025-10-16-swiatowy-dzien-walki-z-glodem>, Dostęp zdalny: (16.06.2026), 2025.
- [3] S. P. Mohanty, D. P. Hughes i M. Salathé, „Using Deep Learning for Image-Based Plant Disease Detection”, *Frontiers in Plant Science*, t. 7, 2016.
- [4] M. E. H. Chowdhury i in., „Automatic and Reliable Leaf Disease Detection Using Deep Learning Techniques”, *AgriEngineering*, t. 3, 2021.
- [5] K. G. Liakos, P. Busato, D. Moshou, S. Pearson i D. Bochtis, „Machine Learning in Agriculture: A Review”, *Sensors (Basel)*, t. 18, nr. 8, s. 2674, sierp. 2018. DOI: 10.3390/s18082674. adr.: <https://doi.org/10.3390/s18082674>.
- [6] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat i N. Batra, *PlantDoc: A Dataset for Visual Plant Disease Detection*, Dostęp zdalny (08.06.2025): <https://doi.org/10.1145/3371158.3371196>, Hyderabad, India, 2020. DOI: 10.1145/3371158.3371196.
- [7] T. O. Emmanuel, *PlantVillage Dataset*, Dostęp zdalny (08.06.2025): <https://www.kaggle.com/datasets/emmarex/plantdisease/data?select=PlantVillage>, 2018.
- [8] S. Bhattarai, *New Plant Diseases Dataset*, Dostęp zdalny (08.06.2025): <https://www.kaggle.com/datasets/vipooooool/new-plant-diseases-dataset>, 2018.
- [9] P. AB, *Planta: Plant & Garden Care*, Dostęp zdalny (08.06.2025): <https://play.google.com/store/apps/details?id=com.stromming.planta>, 2025.
- [10] S. Ltd, *Agrio - Plant diagnosis app*, Dostęp zdalny (08.06.2025): <https://play.google.com/store/apps/details?id=com.agrio>, 2025.
- [11] PlantNet, *PlantNet Plant Identification*, Dostęp zdalny (08.06.2025): <https://play.google.com/store/apps/details?id=org.plantnet>, 2025.
- [12] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997.
- [13] Y. LeCun, Y. Bengio i G. Hinton, „Deep learning”, *nature*, t. 521, nr. 7553, s. 436–444, 2015.
- [14] TensorFlow Agents Team. „Introduction to RL and Deep Q Networks”. Dostęp zdalny (2026-05-19). adr.: https://www.tensorflow.org/agents/tutorials/0_intro_rl.
- [15] I. Goodfellow, Y. Bengio i A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.
- [16] X. Ying, „An Overview of Overfitting and its Solutions”, *Journal of Physics: Conference Series*, t. 1168, nr. 2, s. 022 022, lut. 2019. DOI: 10.1088/1742-6596/1168/2/022022. adr.: <https://doi.org/10.1088/1742-6596/1168/2/022022>.

- [17] C. Cortes i V. Vapnik, „Support-vector networks”, *Machine Learning*, t. 20, s. 273–297, wrz. 1995. DOI: 10.1007/BF00994018. adr.: <https://doi.org/10.1007/BF00994018>.
- [18] A. Horzyk, *Support Vector Machines (SVM)*, <https://home.agh.edu.pl/~horzyk/lectures/miw/MIW-SVM.pdf>, Dostęp zdalny (2026-05-19), 2026.
- [19] GeeksforGeeks, *Linear Regression in Machine learning*, Dostęp zdalny (24.08.2026), 2026. adr.: <https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/>.
- [20] I. Mienye i N. Jere, „A Survey of Decision Trees: Concepts, Algorithms, and Applications”, *IEEE Access*, t. PP, s. 1–1, sty. 2024. DOI: 10.1109/ACCESS.2024.3416838.
- [21] M. Suyal i P. Goyal, „A review on analysis of k-nearest neighbor classification machine learning algorithms based on supervised learning”, *International Journal of Engineering Trends and Technology*, t. 70, nr. 7, s. 43–48, 2022.
- [22] A. Horzyk, *Sztuczne Sieci Neuronowe MLP*, <https://home.agh.edu.pl/~horzyk/lectures/miw/MIW-SztuczneSieciNeuronoweMLP.pdf>, Dostęp zdalny (2026-05-19), 2026.
- [23] K. Kaczmariski. „MiNI Wykłady”. adr.: <https://pages.mini.pw.edu.pl/~kaczmariskik/MiNIWyklady/index.html>.
- [24] GeeksforGeeks. „Multi-Layer Perceptron Learning in TensorFlow”. Dostęp zdalny (2026-05-19). adr.: <https://www.geeksforgeeks.org/deep-learning/multi-layer-perceptron-learning-in-tensorflow/>.
- [25] K. He, X. Zhang, S. Ren i J. Sun, „Deep residual learning for image recognition”, w *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, s. 770–778.
- [26] KSolves. „Understanding Convolution Neural Network Architecture”. Dostęp zdalny (2026-05-19). adr.: <https://www.ksolves.com/blog/artificial-intelligence/understanding-convolution-neural-network-architecture>.
- [27] K. O’Shea i R. Nash, „An Introduction to Convolutional Neural Networks”, *CoRR*, t. abs/1511.08458, 2015. arXiv: 1511.08458. adr.: <http://arxiv.org/abs/1511.08458>.
- [28] A. Vaswani i in., „Attention Is All You Need”, *CoRR*, t. abs/1706.03762, 2017. arXiv: 1706.03762. adr.: <http://arxiv.org/abs/1706.03762>.
- [29] R. Kumar. „Recurrent Neural Network (RNN)”. Dostęp zdalny (2026-05-19). adr.: <https://medium.com/@RobuRishabh/recurrent-neural-network-rnn-8412b9abd755>.
- [30] M. Grandini, E. Bagli i G. Visani, „Metrics for multi-class classification: an overview”, *arXiv preprint arXiv:2008.05756*, 2020.
- [31] D. Alghazzawi, O. Rabie, O. Bamasaq, A. Albeshri i M. Asghar, „Sensor-Based Human Activity Recognition in Smart Homes Using Depthwise Separable Convolutions”, *Human-centric Computing and Information Sciences*, t. 12, paź. 2022. DOI: 10.22967/HCIS.2022.12.050.

- [32] A. G. Howard, „Mobilenets: Efficient convolutional neural networks for mobile vision applications”, *arXiv preprint arXiv:1704.04861*, 2017.
- [33] A. Dosovitskiy, „An image is worth 16x16 words: Transformers for image recognition at scale”, *arXiv preprint arXiv:2010.11929*, 2020.
- [34] M. Tan i Q. Le, „Efficientnet: Rethinking model scaling for convolutional neural networks”, w *International conference on machine learning*, PMLR, 2019, s. 6105–6114.
- [35] G. Huang, Z. Liu, L. van der Maaten i K. Q. Weinberger, *Densely Connected Convolutional Networks*, 2018. arXiv: 1608.06993 [cs.CV]. adr.: <https://arxiv.org/abs/1608.06993>.
- [36] NVIDIA, *Train With Mixed Precision*, Dostęp zdalny (25.08.2026), 2026. adr.: <https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html>.

Wykaz symboli i skrótów

EiTI – Wydział Elektroniki i Technik Informacyjnych

PW – Politechnika Warszawska

WEIRD – ang. *Western, Educated, Industrialized, Rich and Democratic*

Spis rysunków

2.1	Przykładowe zdjęcia ze zbioru danych PlantDoc. [6]	15
2.2	Przykładowe zdjęcia ze zbioru danych PlantVillage.[7]	15
2.3	Przykładowe zdjęcia ze zbioru danych New Plant Diseases.[8]	16
2.4	Przykładowe zrzuty ekranu z aplikacji Planta[9].	17
2.5	Przykładowe zrzuty ekranu z aplikacji Agrio[10].	18
2.6	Przykładowe zrzuty ekranu z aplikacji[11].	19
3.1	Schemat ilustrujący działanie algorytmu SVM	22
3.2	Schemat budowy drzewa decyzyjnego [20]	23
3.3	Dane przed wykorzystaniem algorytmu k-NN	24
3.4	Dane po wykorzystaniu algorytmu k-NN	25
3.5	Schemat budowy neuronu[23]	26
3.6	Schemat budowy perceptronu [23]	27
3.7	Schemat ilustrujący budowę MLP.	27
3.8	Schemat budowy CNN.[26]	28
3.9	Przedstawienie jednostki rekurencyjnej.[13]	30
3.10	Przedstawienie rozwijania RNN	31
3.11	Przykładowa macierz pomyłek dla trzech klas, z wynikami otrzymywanymi dla klasy ‘B’.	32
4.1	Przedstawienie działania konwolucji głębokościowej.[31]	35
4.2	Przedstawienie budowy oraz działania ViT.[33]	36
4.3	Przedstawienie budowy oraz działania bloku rezyduального.[25]	38
4.4	Skalowanie modelu (a) to bazowy przykład modeli; (b)-(d) są modelami, w których zwiększono tylko jeden wymiar; (e) to model w ramach, którego zastosowano skalowanie złożone.[34]	39
4.5	Struktura modelu DenseNet. [35]	40
5.1	Przykładowe zdjęcia ze zbioru danych. [7]	41
6.1	Przykładowe zdjęcia ze zbioru danych [7] na których dokonano przekształceń.	45
6.2	Przykładowe zdjęcie ze zbioru danych [7] po poddaniu go augmentacji danych.	47
9.1	Zdjęcie ekranu startowego oraz ekranu z wynikami	59

Spis tabel

5.1	Statystyki liczby zdjęć w zbiorze danych	42
6.1	Lista modeli wykorzystanych w badaniu	44
6.2	Lista zbiorów oraz liczba zdjęć	45
6.3	Operacje i parametry stosowane podczas przygotowania danych treningowych	48
7.1	Statystyki modeli dla tego samego zbioru danych	52
7.2	Wyniki klasyfikacji dla zbiorów treningowych o różnej wielkości	52
7.3	Wyniki klasyfikacji dla różnego procenta zaszumienia zbioru treningowego . .	53

Spis załączników

1.	Link do repozytorium GitHub	67
----	---------------------------------------	----

Załącznik 1. Link do repozytorium GitHub

https://github.com/JakubKostrzewa20/praca_dyplomowa_magisterskie